

LAB MANUAL



Creation of Logic apps and Endpoint

COM682

Practical Lab-08

**Faculty of Computing, Engineering and The
Built Environment**

Contents

1. Background.....	3
2.1 Table Design.....	3
2.2 JSON Design.....	9
2.3 REST API Implementation.....	10
Creating a Logic App endpoint (Consumption mode).....	10
Creating a GET REST endpoint to obtain all records (RAA).....	12
1. Creating a POST endpoint (RAA)	14
2. Creating a POST endpoint (CIA)	22
3. Creating a DELETE endpoint (DIA)	28
4. Creating a GET endpoint to retrieve a specific record (RIA)	32
5. Creating a PUT endpoint to UPDATE a specific record (UIA)	34
Appendix I – Testing a REST API.....	39
Get all assets	39
POST a new asset.....	39
DELETE a stored asset	40
GET an individual record.....	42
PUT/Update an individual record	43

1. Background

In this week's lab, you will create REST APIs to CREATE, READ, UPDATE and DELETE the information at your storage account.

We will first create a DB instance, then will create 5 Logic Apps for :

- Creation of a new asset record (**CNA**)
- Retrieval of all asset records (**RAA**)
- Retrieval of individual asset records (**RIA**)
- Updating an asset record (**UIA**)
- Deletion of an asset record (**DIA**)

2.1 Table Design

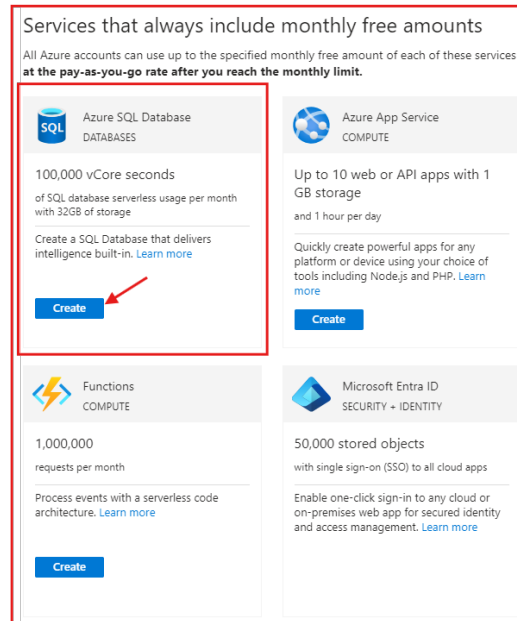
This solution needs only a single table to store all information required. This is due to its simplistic, initial, design.

This Table is to be called “assetTracking”, the data to be stored in it is pictured below.

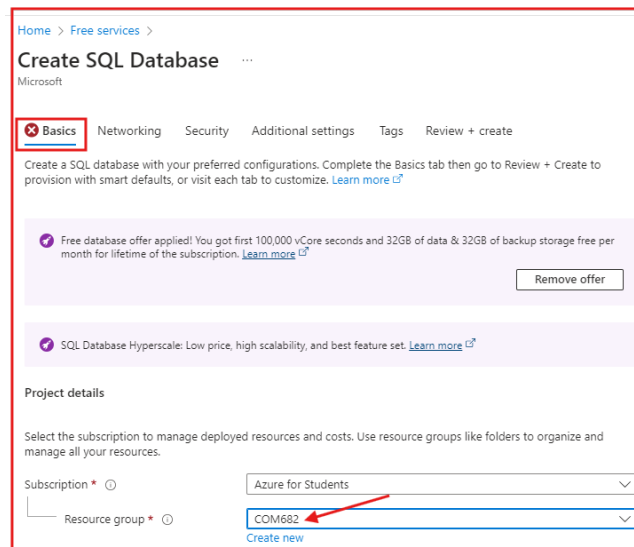
<i>Field Name</i>	<i>Data Type</i>	<i>Description</i>
AssetID	BigInt, Autoincrementing	An ID for each asset tuple
AssetName	Varchar(255)	A label for the asset
Cost	Integer	The valuation in roubles
AssetType	Varchar(255)	A label for the asset type
NameOfOwner	Varchar(255)	The Name of the current owner
AddressLine1	Varchar(255)	The first line of an address
AddressLine2	Varchar(255)	The second line of its address
Note	Text	A miscellaneous note

Now the database schema has been designed, there is a need to create a hosted database which can host this table and service a REST API.

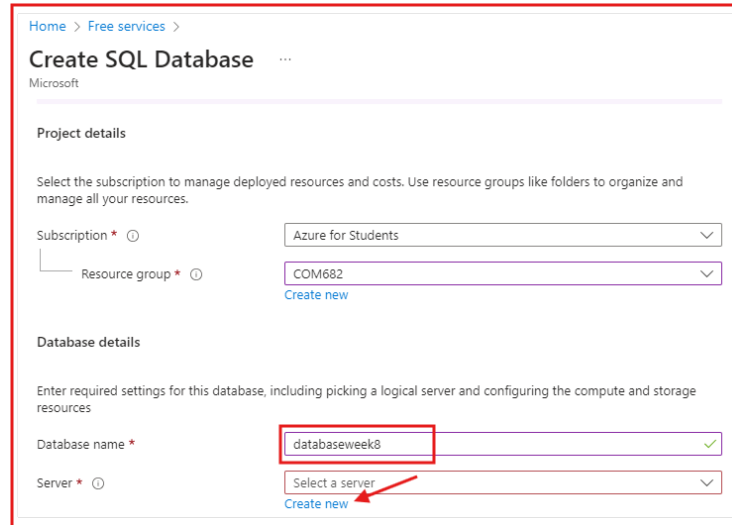
To start with, go to free services and select “Azure SQL Database” as shown in the image below:



After clicking the create button, the next page you will see will look like the one below:



On the Basic tab, select the subscription and provide the resource group. If you don't already have a resource group, create one by clicking on create new. Next, provide the database details by entering the Database name. For this lab, I provided the "databaseweek8", however, you can provide any name of your choice. For server information, click on the "create new" to create a new server.



Home > Free services >

Create SQL Database

Microsoft

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ Azure for Students ▼

Resource group * ⓘ COM682 ▼

[Create new](#)

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name * databaseweek8 ✓

Server * ⓘ Select a server ▼

[Create new](#)

Start to setup a SQL instance as has been outlined in previous practical exercises.

- In this instance we will call this database **assettrak-bXXX-com682** (Where bXXX is your B number) and make its location according to your allocated region. In my case, I provided the server name as **serverweek8** and location as **(Europe) UK South**.
- Set authentication to use SQL authentication.
- Set a memorable username and password. We have set the admin login to **week8server** with the password of **Labuser1**, this is presented below. Once done, click OK.

Ensure you take a note of your server name, server admin login, and password information. You **will** need this later in the tutorial.

Home > Free services > Create SQL Database >

Create SQL Database Server

Microsoft

Server details

Enter required settings for this server, including providing a name and location. This server will be created in the same subscription and resource group as your database.

Server name * ✓

Location * ✓

Authentication

[i](#) Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#) ^U

Select your preferred authentication methods for accessing this server. Create a server admin login and password to access your server with SQL authentication, select only Microsoft Entra authentication [Learn more](#) ^U using an existing Microsoft Entra user, group, or application as Microsoft Entra admin [Learn more](#) ^U, or select both SQL and Microsoft Entra authentication.

Authentication method

- ☐ Use Microsoft Entra-only authentication
- ☐ Use both SQL and Microsoft Entra authentication
- ☒ Use SQL authentication

Server admin login * ✓

Password * ✓

Confirm password * ✓

Click Configure database and update the configuration to reflect free tier as appropriate.

Home > Free services >

Create SQL Database

Microsoft

Database details

Enter required settings for this database, including picking a logical server and configuring the compute and storage resources

Database name * ✓

Server * ✓

[Create new](#)

Compute + storage * ✓

Standard-series (Gen5), 2 vCores, 32 GB storage, zone redundant disabled

[Configure database](#)

Behavior when free offer limit reached

Behavior when free offer limit reached ☐ Auto-pause the database until next month

When free offer limit is reached, the database will not be accessible until the beginning of next calendar month when free amount is renewed. There will be no additional charges.

☒ Continue using database for additional charges

Database continues to be accessible after free offer limit is reached. Additional usage beyond the free offer amount for that month will be charged at general purpose serverless tier rates. The free amount will be renewed at the beginning of the next calendar month.

Warning: If you choose to continue using the database for additional charges, your database will continue running after your free offer limit is reached and you will be charged for any overages during each calendar month. You will not be able to restore the auto-pause setting for this database, however you will still continue to receive the free offer amount each month. [Learn more](#) ^U

Configure as appropriate for the current free tier offering (this may deviate from the screenshots below) and click Apply.

Home > Free services >

Configure

[Feedback](#)

Service and compute tier

Select from the available tiers based on the needs of your workload. The vCore model provides a wide range of configuration controls and offers Hyperscale and Serverless to automatically scale your database based on your workload needs. Alternately, the DTU model provides set price/performance packages to choose from for easy configuration. [Learn more](#)

Free database offer (Preview) ☒ Applied

Behavior when free offer limit reached

☐ Auto-pause the database until next month
When free offer limit is reached, the database will not be accessible until the beginning of next calendar month when free amount is renewed. There will be no additional charges.

☒ Continue using database for additional charges
Database continues to be accessible after free offer limit is reached. Additional usage beyond the free offer amount for that month will be charged at general purpose serverless tier rates. The free amount will be renewed at the beginning of the next calendar month.

⚠️ If you choose to continue using the database for additional charges, your database will continue running after your free offer limit is reached and you will be charged for any overages during each calendar month. You will not be able to restore the auto-pause setting for this database, however you will still continue to receive the free offer amount each month. [Learn more](#)

☒ SQL Database Hyperscale: Low price, high scalability, and best feature set. [Learn more](#)

Max vCores:

Min vCores:

3.75 GB MIN MEMORY 30 GB MAX MEMORY

Data max size (GB)

9.6 GB LOG SPACE ALLOCATED

Would you like to make this database zone redundant? ☐ Yes ☒ No

[Apply](#)



Cost summary

General Purpose (GP, S, Gen5, 10)

Cost per GB (in USD) 0.13

Max storage selected (in GB) x 41.6

First 32 GB storage free

First 100,000 vCore seconds free

Overage billing¹ Enabled

ESTIMATED STORAGE COST / MONTH 5.50 USD

COMPUTE COST / VCORE SECOND² 0.000181 USD

¹ There will be no charges for usage within the free limits. Overage billing will result in charges for usage exceeding the free limits.

² Serverless databases are billed in vCore seconds based on a combination of CPU and memory utilization. [Learn more about serverless billing](#)

Home > Free services >

Create SQL Database

Microsoft

Database name *

Server *

[Create new](#)

Compute + storage * ☐ **General Purpose - Serverless**
Standard-series (Gen5), 10 vCores, 32 GB storage, zone redundant disabled
[Configure database](#)

Behavior when free offer limit reached

Behavior when free offer limit reached ☐ Auto-pause the database until next month
When free offer limit is reached, the database will not be accessible until the beginning of next calendar month when free amount is renewed. There will be no additional charges.

☒ Continue using database for additional charges
Database continues to be accessible after free offer limit is reached. Additional usage beyond the free offer amount for that month will be charged at general purpose serverless tier rates. The free amount will be renewed at the beginning of the next calendar month.

⚠️ If you choose to continue using the database for additional charges, your database will continue running after your free offer limit is reached and you will be charged for any overages during each calendar month. You will not be able to restore the auto-pause setting for this database, however you will still continue to receive the free offer amount each month. [Learn more](#)

Backup storage redundancy

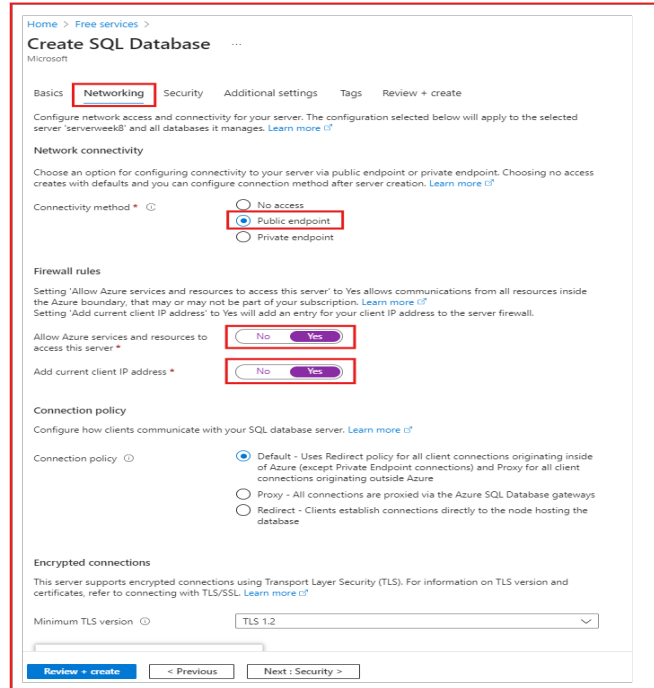
Choose how your PITR and LTR backups are replicated. Geo restore or ability to recover from regional outage is only available when geo-redundant storage is selected.

Backup storage redundancy ☐ Locally-redundant backup storage
☐ Zone-redundant backup storage
☒ Geo-redundant backup storage
☐ Geo-Zone-redundant backup storage

⚠️ Selected value for backup storage redundancy is Geo-redundant backup storage. Database backups will be geo-replicated which might impact your data residency requirements. [Learn more](#)

[Review + create](#) [Next: Networking >](#)

As before, we have ensured that we have access to manage this server from our current IP address and have enabled this to be used as a **public endpoint**. To do this, select the **Next: Networking** button.



Home > Free services > Create SQL Database

Microsoft

Basics **Networking** Security Additional settings Tags Review + create

Configure network access and connectivity for your server. The configuration selected below will apply to the selected server 'serverweek8' and all databases it manages. [Learn more](#)

Network connectivity

Choose an option for configuring connectivity to your server via public endpoint or private endpoint. Choosing no access creates with defaults and you can configure connection method after server creation. [Learn more](#)

Connectivity method *

☐ No access ☒ **Public endpoint** ☐ Private endpoint

Firewall rules

Setting 'Allow Azure services and resources to access this server' to Yes allows communications from all resources inside the Azure boundary, that may or may not be part of your subscription. [Learn more](#)

Setting 'Add current client IP address' to Yes will add an entry for your client IP address to the server firewall.

Allow Azure services and resources to access this server *

☐ No ☒ **Yes**

Add current client IP address *

☐ No ☒ **Yes**

Connection policy

Configure how clients communicate with your SQL database server. [Learn more](#)

Connection policy

☒ **Default** - Uses Redirect policy for all client connections originating inside of Azure (except Private Endpoint connections) and Proxy for all client connections originating outside Azure

☐ Proxy - All connections are proxied via the Azure SQL Database gateways

☐ Redirect - Clients establish connections directly to the node hosting the database

Encrypted connections

This server supports encrypted connections using Transport Layer Security (TLS). For information on TLS version and certificates, refer to connecting with TLS/SSL. [Learn more](#)

Minimum TLS version

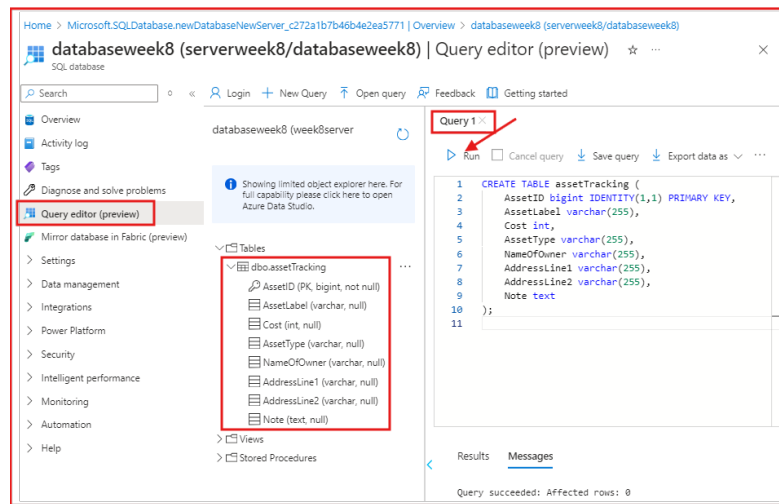
[Review + create](#) < Previous Next: Security >

Leave the rest of the options (Security, Additional Settings, and Tags) as it is (default), and click on the Review + Create to create the Database. Once the database has been created, open the online query editor. Then issue an SQL command to create the previously designed table. The SQL command used is presented below.

Note the use of the IDENTITY keyword to have an auto-incrementing, big integer based, primary key.

```
CREATE TABLE assetTracking (
    AssetID bigint IDENTITY(1,1) PRIMARY KEY,
    AssetLabel varchar(255),
    Cost int,
    AssetType varchar(255),
    NameOfOwner varchar(255),
    AddressLine1 varchar(255),
    AddressLine2 varchar(255),
    Note text
);
```


The execution of this is presented below.



As a table has been established, we must not model the data exchange format concerning identities within that table. This is the focus of the next subsection (2.2).

2.2 JSON Design

The records in the designed table will be transited to and from consuming software over REST and in the JSON format. The generic document to be interchanged in these transactions will follow the design of the table and is presented below.

```
1 {
2   "AssetID": 1,
3   "AssetLabel": "Terrance and Philip defend Canada",
4   "Cost": 69,
5   "AssetType": "Painting",
6   "NameOfOwner": "Trey Parker",
7   "AddressLine1": "1600 Pennsylvania Avenue",
8   "AddressLine2": "Washington",
9   "Note": "Loaned from Trey to yer man from the telly"
10 }
```

In order to best support this solutions growth, we are going to deploy a REST API which has native consideration of versioning in the specified relative path. For this exercise we will develop version 1 of the API.

This REST endpoint needs to support the following functions:

- Retrieval of all asset records (**RAA**)
- Creation of a new asset record (**CIA**)
- Retrieval of individual asset records (**RIA**)
- Updating an asset record (**UIA**)
- Deletion of an asset record (**DIA**)

These functions should be mapped onto a versioned REST URI schema. This is presented in the below table.

<i>Function</i>	<i>Path</i>	<i>HTTP Verb</i>
Entity Name: Asset		
RAA	/rest/v1/assets	GET
CIA	/rest/v1/assets	POST
RIA	/rest/v1/assets/{id}	GET
DIA	/rest/v1/assets/{id}	DELETE
UIA	/rest/v1/assets/{id}	PUT

This table is essential in later procedures where the HTTP method/verb is defined in addition to the relative path.

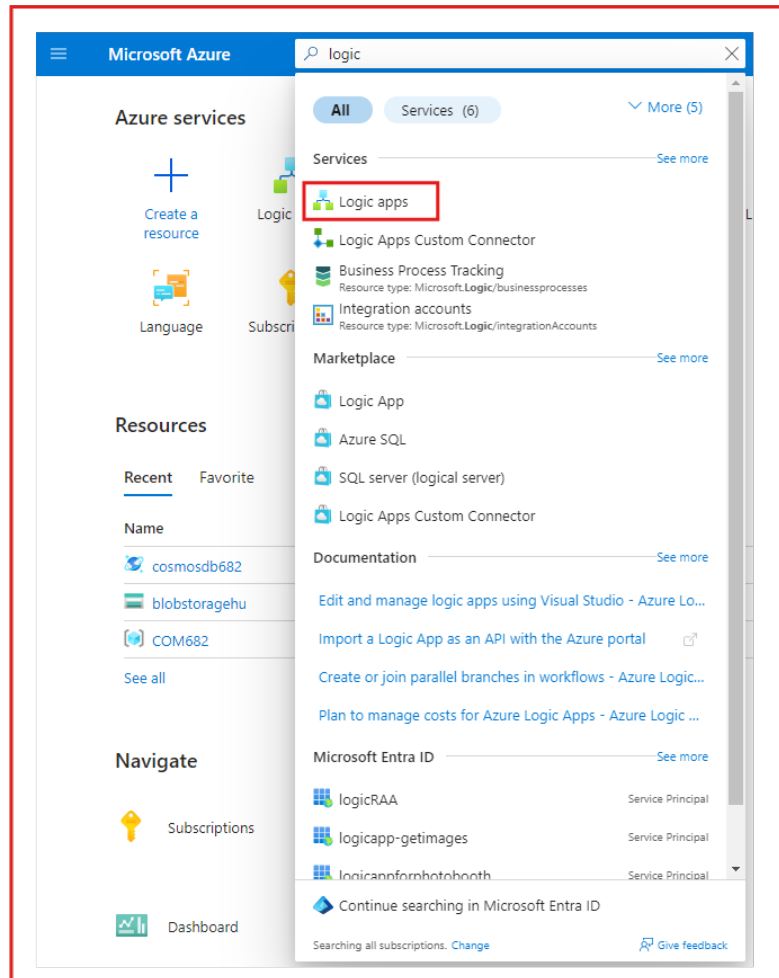
Once the general design of the REST API and JSON format has been solidified, it is now possible to implement the APIs. For this solution, we will be using a no code/low code solution in the form of Azure Logic Apps. This will be detailed in the following subsection (2.3).

2.3 REST API Implementation

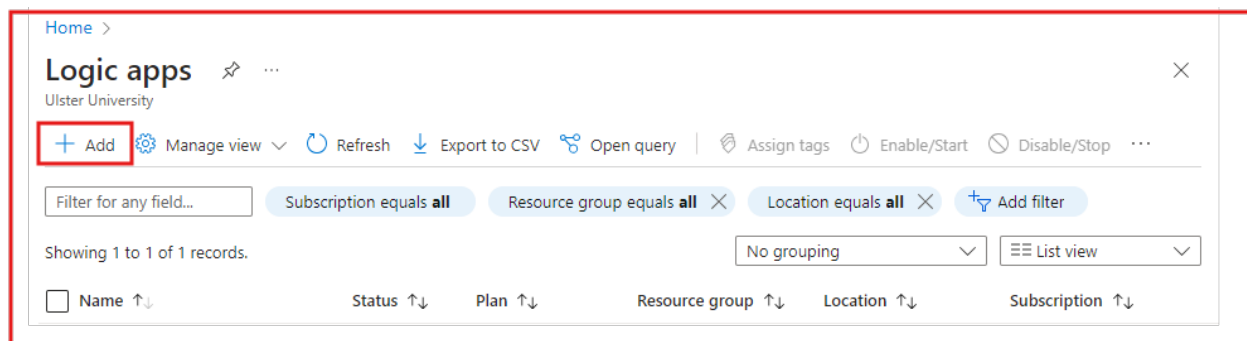
In this practical we will be creating logic apps. These are graphically defined low-code/no-code mechanisms which may produce REST endpoints, among other features.

Creating a Logic App endpoint (Consumption mode)

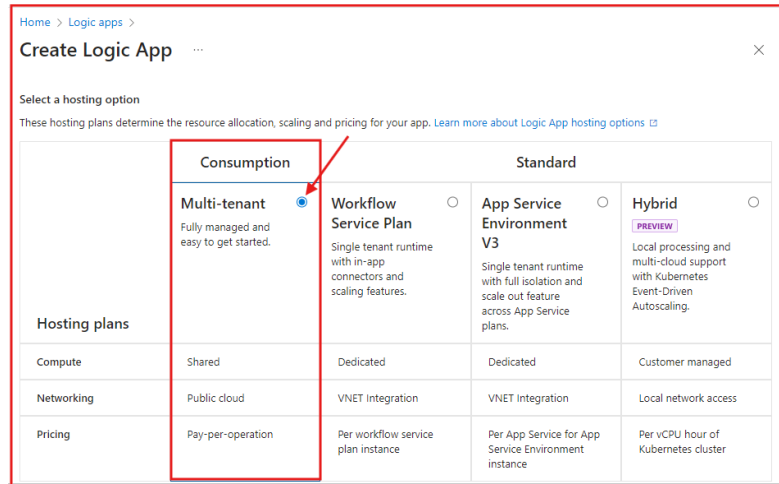
This section describes creating a Logic app to serve as the rest endpoint host. This should be created from the Free services index. Search for Logic Apps in the Azure Search Bar and select the Logic app as shown in the image below.



It will open a new window, click on the +Add to add a new logic App as shown in the image.



Next Select the Multi-Tenant option under the Consumption as shown below.



Create Logic App (Multi-tenant) window will open where you will need to provide the following information as shown in the image below.

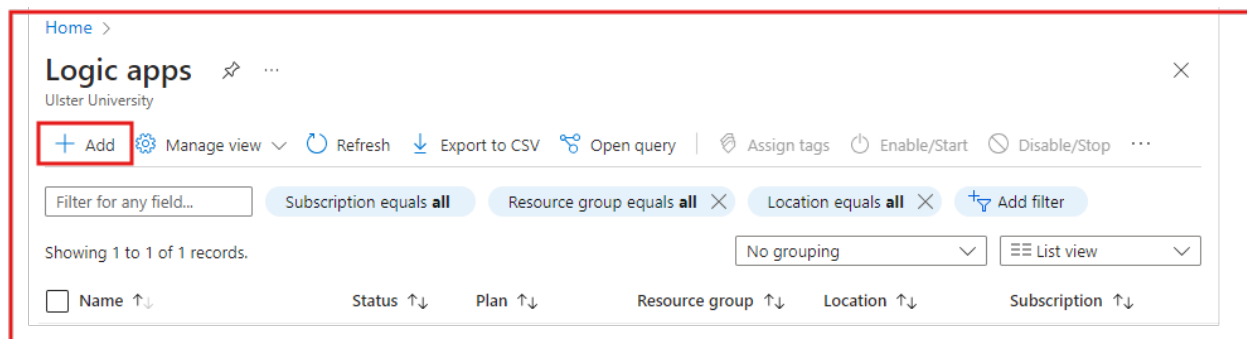
Resource Group	COM682 (select any resource group you created previously)
Logic App Name	RAA
Region	West US (use location available to you)
Tags	Provide any tags if you want

Next, Click on Review + Create button and finally click on Create to create the logic App as shown below.

Creating a GET REST endpoint to obtain all records (RAA)

In this example we are going to deploy the RAA endpoint.

To do this, click on the +Add to add a new logic App as shown in the image.



Next Select the Multi-Tenant option under the Consumption as shown below.

Home > Logic apps > Create Logic App >

Create Logic App (Multi-tenant) ...

Basics Tags Review + create

Create a logic app, which lets you group workflows as a logical unit for easier management, deployment and sharing of resources. Workflows let you connect your business-critical apps and services with Azure Logic Apps, automating your workflows without writing a single line of code.

Project Details

Select a subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription *

Resource Group * [Create new](#)

Instance Details

Logic App name *

Region *

Enable log analytics * ☐ Yes ☒ No

⚠ There are no log analytics workspace resources in the selected subscription. In order to enable log analytics, either create a new log analytics workspace resource or switch to a subscription which already has one.

ℹ Looking for the classic consumption create experience? [Click here](#)

Finally, Click Review + Create to create the RAA endpoint.

Repeat this step for each of the endpoints (CIA, DIA, RIA, UIA). This will provide endpoints to be defined later in this process.

Due to many users wish to use this app name, you may need to provide your B Number and COM module number in this process, i.e., instead of DIA you may specify DIA-BXX-COM682 (where BXX is your student number). Note your alternative names.

Home > Logic apps ...

Ulster University (ulster.ac.uk)

+ Add ⚙ Manage view ↕ Refresh ⬇ Export to CSV 🔗 Open query | 🏷 Assign tags ⏻ Enable/Start ⏻ Disable/Stop 🗑 Delete

Filter for any field... Subscription equals all Resource group equals all X Location equals all X Add filter

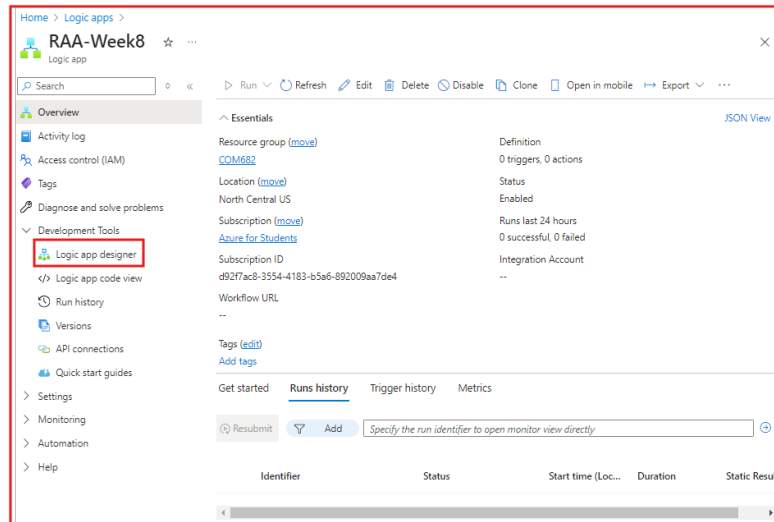
Showing 1 to 8 of 8 records. Group by location List view

☐ Name ↑↓	Status ↑↓	Plan ↑↓	Resource group ↑↓	Location ↑↓	Subscription ↑↓	
▼ North Central US						
☐ CIA-week8	Enabled	Consumption	COM682	North Central US	Azure for Students	...
☐ DIA-week8	Enabled	Consumption	COM682	North Central US	Azure for Students	...
☐ RAA-Week8	Enabled	Consumption	COM682	North Central US	Azure for Students	...
☐ RIA-week8	Enabled	Consumption	COM682	North Central US	Azure for Students	...
☐ UIA-week8	Enabled	Consumption	COM682	North Central US	Azure for Students	...

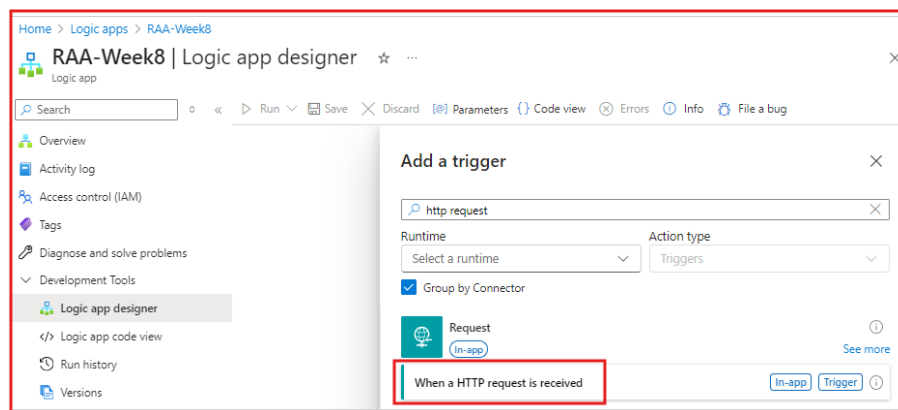
Once these have been initiated, it is now possible to design these API endpoints using the workflows/Azure Logic.

1. Creating a POST endpoint (RAA)

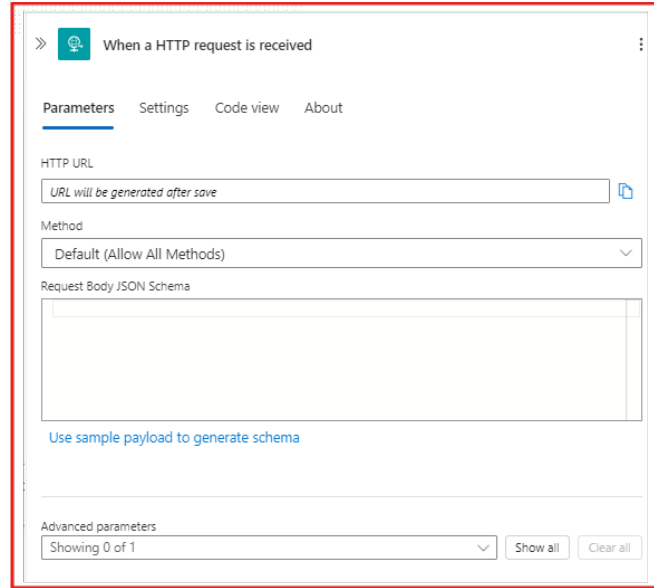
In the Logic apps view, select RAA and click on **Logic App designer** under the deployment tools as shown below.



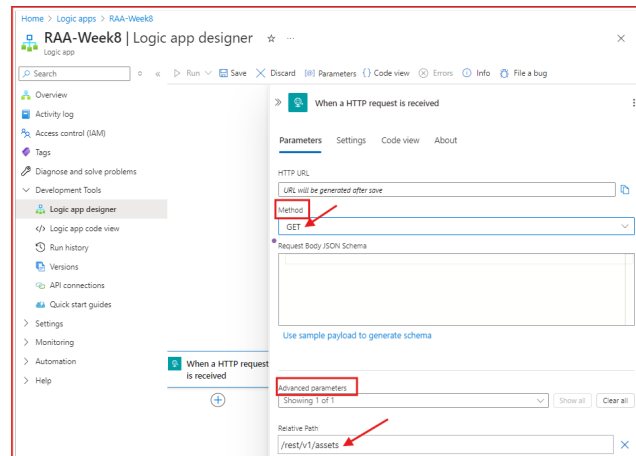
You must now choose an initial operation (trigger) suitable for your app type. As this is a REST endpoint, select “When a HTTP request is received”. To do that, click on the “Add a trigger”, and in the search bar enter “HTTP Request” and select “When an HTTP Request is received” as shown in the image below.



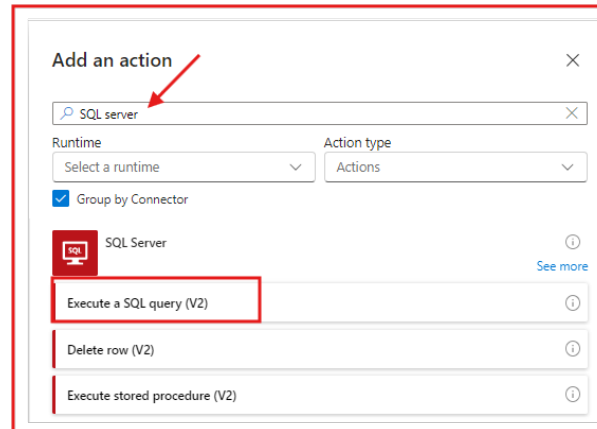
A template HTTP request workflow will be presented which will ask you to provide your request JSON body Schema.



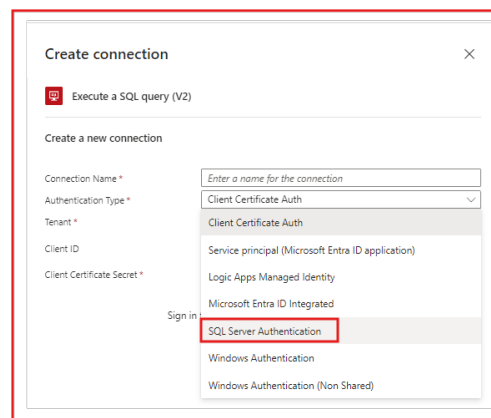
As this is a GET operation it does not require a JSON payload. Subsequently, specify additional parameters in the form of Methods (HTTP Verb) and relative paths. From the drop-down menu under the Method, select “GET”, and for relative path, tick the Relative path under the advance parameters and then provide the relative path as shown below:



Next, click on the + button to connect this HTTP request to a database and issue a query. In this case we will specify the **Execute a SQL query (V2)** command.



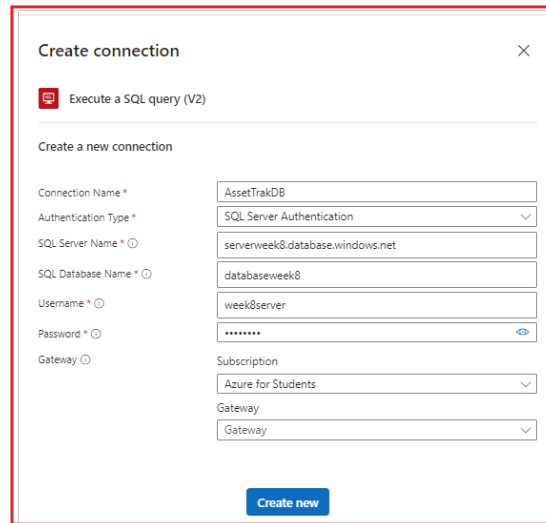
The SQL server object will request you configure it to access the database which was created in this tutorial. By default it will set Authentication type to Client Certificate Auth, however, in this approach we are going to use SQL server authentication.



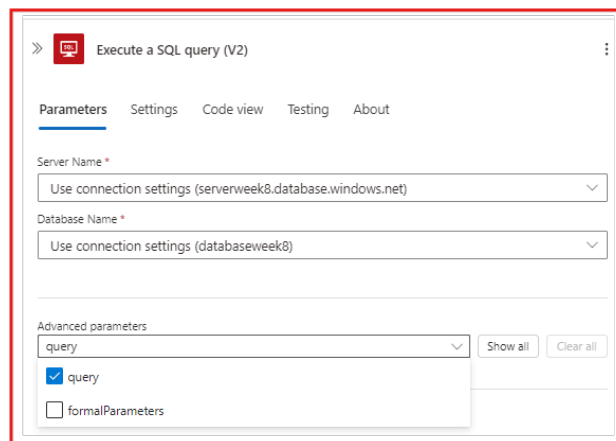
You will be asked to provide some information in order to configure the database. This includes:

1. The connection name, this is meaningful connection name for your own reference. In this tutorial it has been set to **AssetTrakDB**
2. The SQL server name, this is the name of the SQL server you created previously in this tutorial. If you don't recall this then you can view it in the Database interface, under Overview. In this case this was set to **serverweek8.database.windows.net**
3. The name of the SQL database, in this case it is **databaseweek8**
4. The Username and password combination, **week8server** and **Labuser1** respectively (THESE ARE TERRIBLE CREDENTIALS)

Provide this information and create the object.



In the next interface you will be asked to confirm the SQL server name and database from a drop down list. You can then add a new parameter and select query

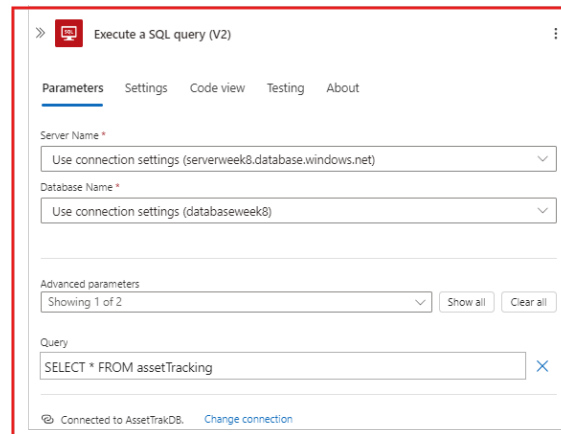


If you find error here, go to DB->networking-> Exceptions 'Allow Azure services and resources to access this server.

Specifying this parameter provides a query text box. This provides the ability to specify a SQL query to be executed.

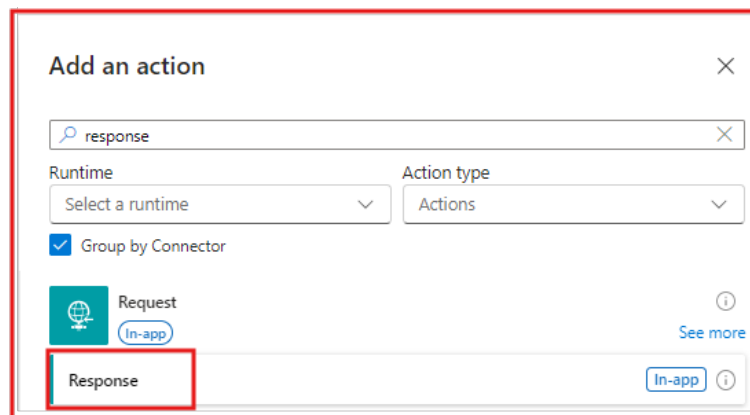
In this case we wish to execute a simple sql query to return all the asset records. The query used in this case is:

```
SELECT * FROM assetTracking
```

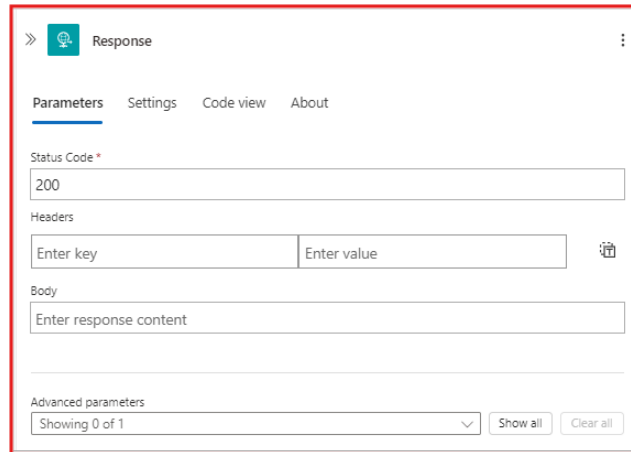


This is the only time in this tutorial that we use this query execution interface. We use it in this case to make the reader aware that this powerful feature is a possibility.

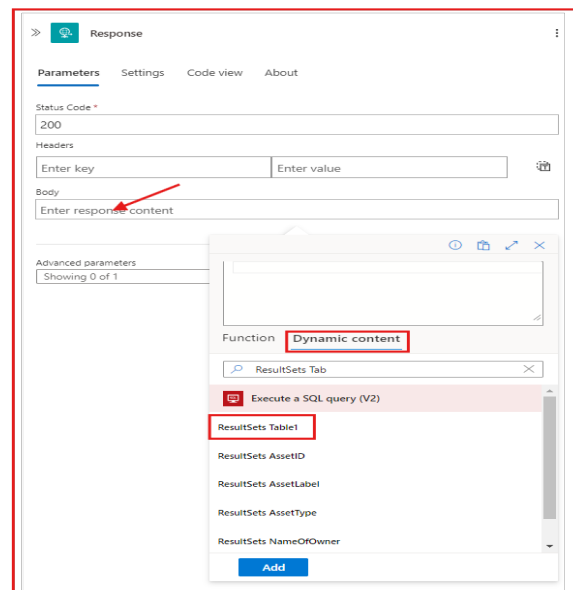
Next, another action needs to be chained onto this workflow. This is to provide a response to the client, so it doesn't time out. Select a Response/request element.



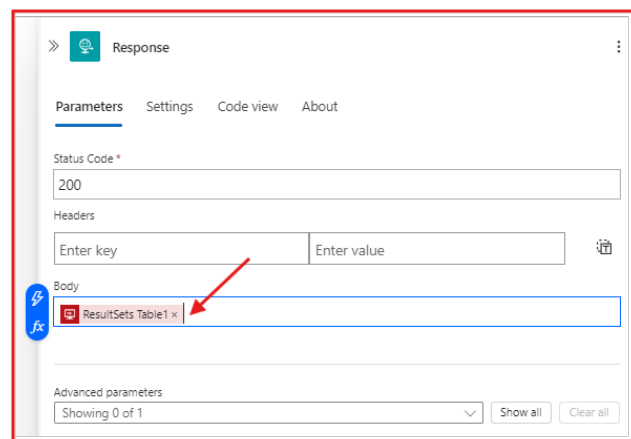
This will then create the response/request object.



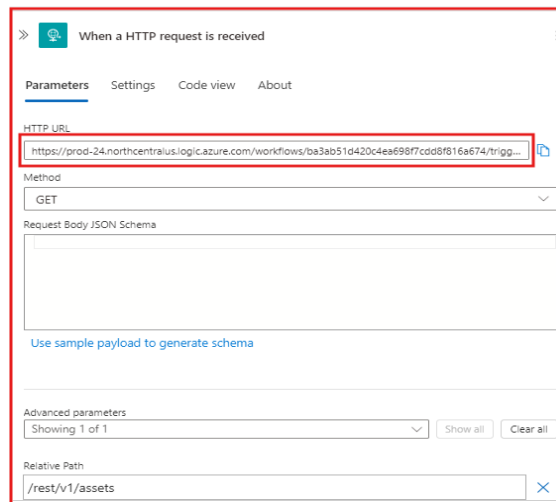
Select body and locate the SQL server **ResultSets Table1** Item under the *Execute a SQL Query (V2)* area. This will return the output from that query and automatically provide this as JSON.



Once this has been selected then you may press Save.



Congratulations, you have established a REST API endpoint. Next it would be prudent to test this. Obtain the URL of the endpoint, do this by navigating to the HTTP request received block and copy the GET URL.



In this case, this was:

[https://prod-24.northcentralus.logic.azure.com/workflows/ba3ab51d420c4ea698f7cdd8f816a674/triggers/When a HTTP request is received/paths/invoke/rest/v1/assets?api-version=2016-10-01&sp=%2Ftriggers%2FWhen a HTTP request is received%2Frun&sv=1.0&sig=bnU1_yktxCRt0D4p67Pq8WrAQjsEEGIYTtq4KcgbDOU](https://prod-24.northcentralus.logic.azure.com/workflows/ba3ab51d420c4ea698f7cdd8f816a674/triggers/When_a_HTTP_request_is_received/paths/invoke/rest/v1/assets?api-version=2016-10-01&sp=%2Ftriggers%2FWhen_a_HTTP_request_is_received%2Frun&sv=1.0&sig=bnU1_yktxCRt0D4p67Pq8WrAQjsEEGIYTtq4KcgbDOU)

Note: this URL has an API key in it to help secure access to your endpoint (SIG).

In a separate document, maintain a table to map the REST endpoints. This should include the function identifier (RAA etc), the HTTP VERB used, the Relative path and the URI that of the REST endpoint. *This will be needed later when adapting the template webapp.*

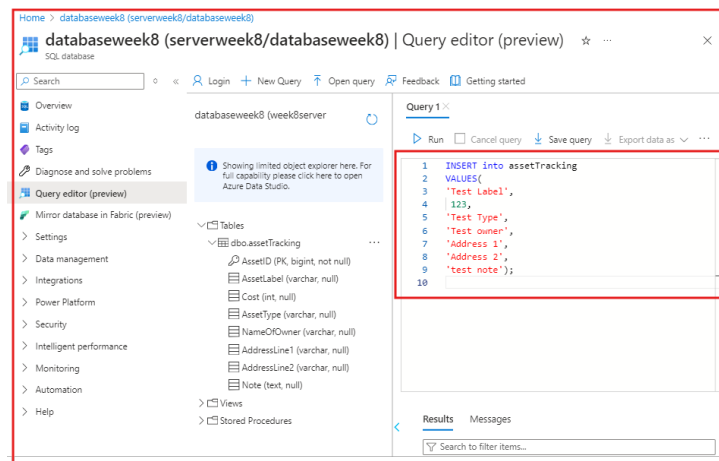
As this URL is of the “GET” variety and permissively configured, it is possible to view it in a web browser. In the screenshot below this returns an empty array as there is no data in that table.



In order to be assured that this is functioning correctly we can insert dummy data.

Using the SQL query editor, we can issue the following SQL statement:

```
INSERT into assetTracking
VALUES(
'Test Label',
123,
'Test Type',
'Test owner',
'Address 1',
'Address 2',
'test note');
```



This alters the contents of the database which subsequently alters the response by the REST endpoint. This is shown below. Note that the results are provided as an array within the master array.

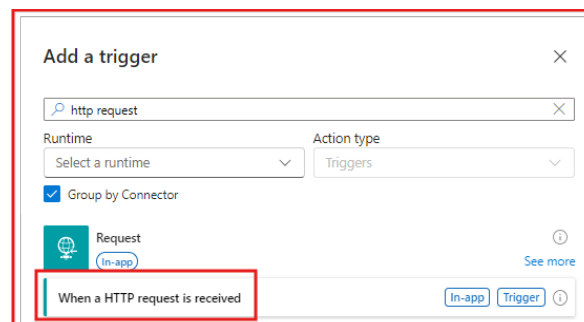


Next, lets create a POST endpoint to enable client to insert data without requiring SQL queries to be directly issued.

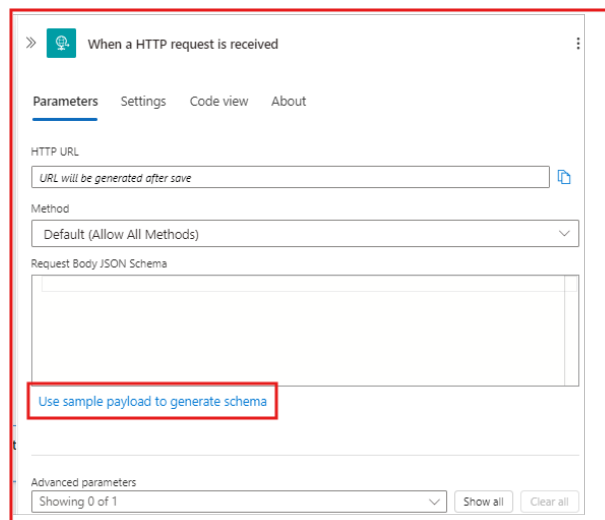
2. Creating a POST endpoint (CIA)

Next, we need to create the CIA endpoint to enable apps/clients to create records using standard REST APIs.

Open the CIA workflow/logic app under the logic app interface. Select the first element as “When a HTTP request is received”, as before.



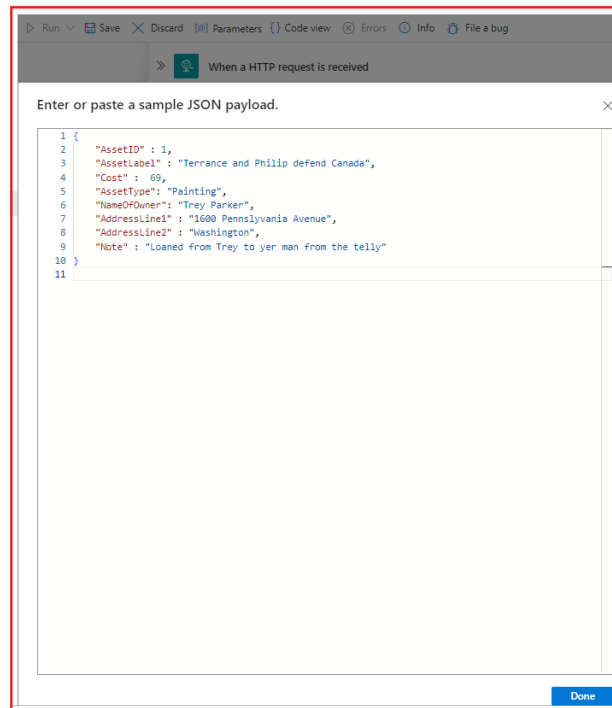
Provide a sample JSON payload, as outlined previously. Once the designer loads, select the “Use sample payload to generate schema” option



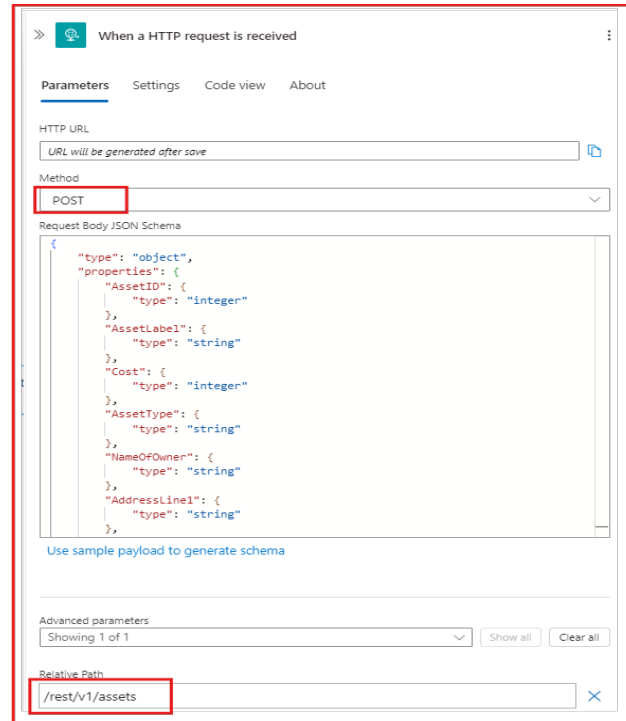
You will be asked to provide some sample JSON, use the example provided in the earlier section of this document:

```
{
  "AssetID" : 1,
  "AssetLabel" : "Terrance and Philip defend Canada",
  "Cost" : 69,
```

```
"AssetType": "Painting",  
"NameOfOwner": "Trey Parker",  
"AddressLine1" : "1600 Pennsylvania Avenue",  
"AddressLine2" : "Washington",  
"Note" : "Loaned from Trey to yer man from the telly"  
}
```



Set the method to post and provide a relative path, as previously defined in the table produced.



When a HTTP request is received

Parameters Settings Code view About

HTTP URL
URL will be generated after save

Method
POST

Request Body JSON Schema

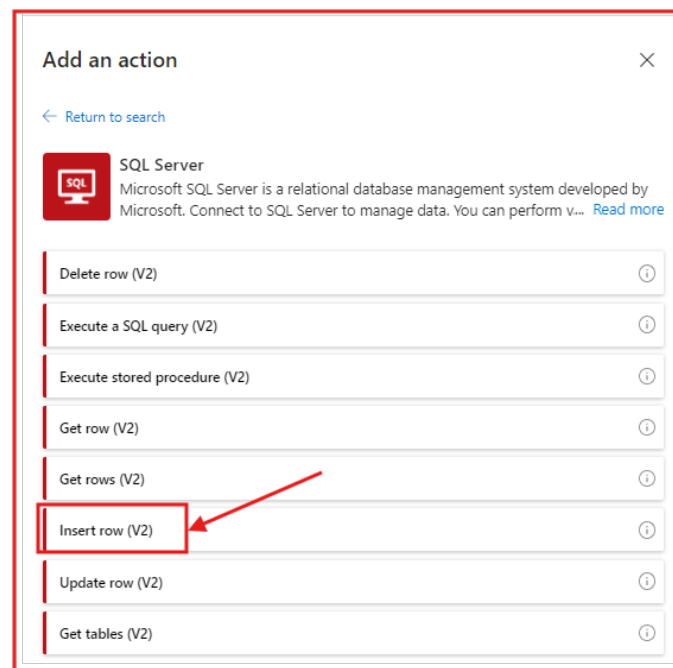
```
{
  "type": "object",
  "properties": {
    "AssetID": {
      "type": "integer"
    },
    "AssetLabel": {
      "type": "string"
    },
    "Cost": {
      "type": "integer"
    },
    "AssetType": {
      "type": "string"
    },
    "NameOfOwner": {
      "type": "string"
    },
    "AddressLine1": {
      "type": "string"
    }
  }
}
```

Use sample payload to generate schema

Advanced parameters
Showing 1 of 1 Show all Clear all

Relative Path
/rest/v1/assets

Next, provide an **Insert Row (V2)** action from a library located in the Azure subsection, opposed to the built-in execute query option provided previously.



Add an action

Return to search

SQL Server
Microsoft SQL Server is a relational database management system developed by Microsoft. Connect to SQL Server to manage data. You can perform v... [Read more](#)

- Delete row (V2)
- Execute a SQL query (V2)
- Execute stored procedure (V2)
- Get row (V2)
- Get rows (V2)
- Insert row (V2)
- Update row (V2)
- Get tables (V2)

You may need to configure the parameters of this object, as you have done with the previous endpoint.

Create connection

×

Execute a SQL query (V2)

Create a new connection

Connection Name *

AssetTrakDB

Authentication Type *

SQL Server Authentication

SQL Server Name * ⓘ

serverweek8.database.windows.net

SQL Database Name * ⓘ

databaseweek8

Username * ⓘ

week8server

Password * ⓘ

Gateway ⓘ

Subscription

Azure for Students

Gateway

Gateway

Create new

Next, provide the server name, Database Name and name of the table you wish to update.

»

Insert row (V2)

Parameters

Settings

Code view

Testing

About

Server Name *

Use connection settings (serverweek8.database.windows.net)

Database Name *

Use connection settings (databaseweek8)

Table Name *

assetTracking

Advanced parameters

Showing 0 of 7

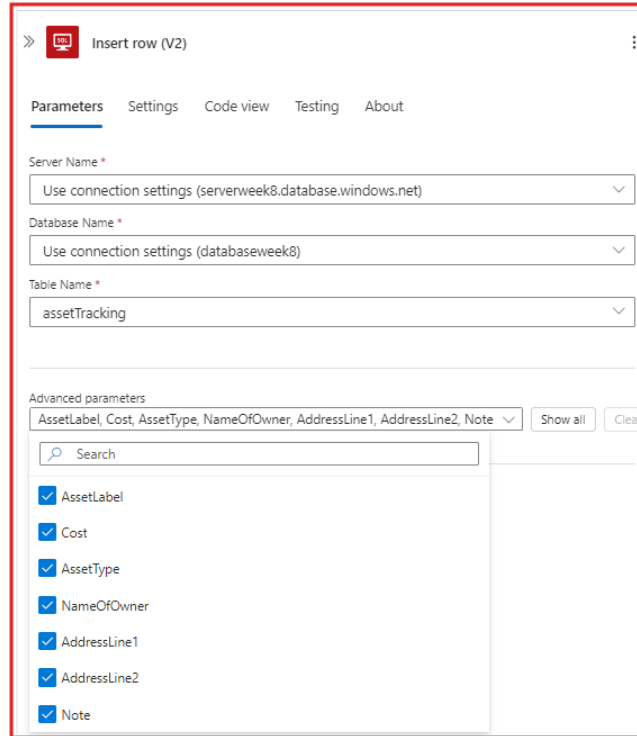
Show all

Clear all

Connected to AssetTrakDB.

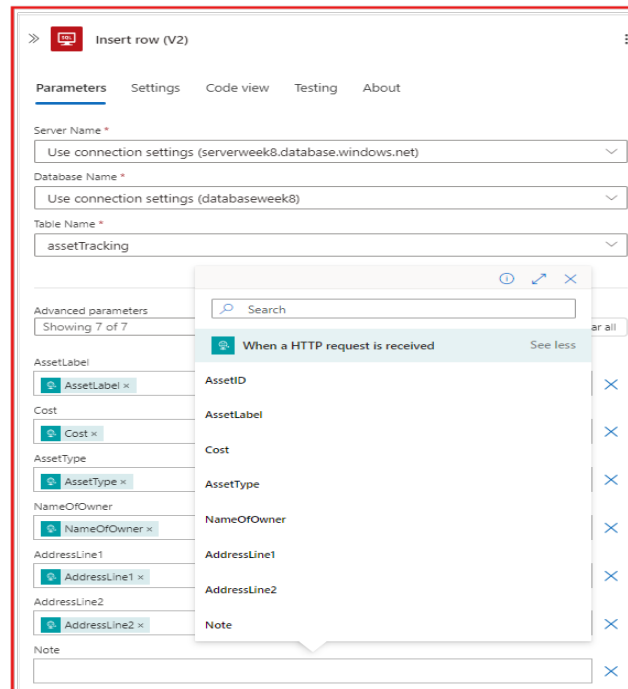
Change connection

Next provide the parameters informed by the provided JSON reference document. This directly ties into what data is stored by the database. In this case, we select all the parameter options.

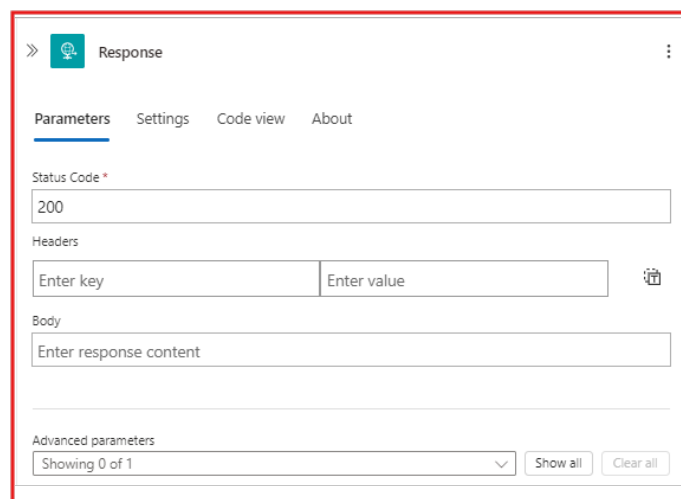


Once the parameters are refined, ensure that they correspond to the correct dynamic content fields i.e the AssetLabel SQL field corresponds to the AssetLabel Key from the JSON schema. This is a graphical operation.

If a field is missing, select the see more option or type to get the element enumerated.



Finally, a HTTP response object is defined. You do not necessarily need to provide a body in reply.

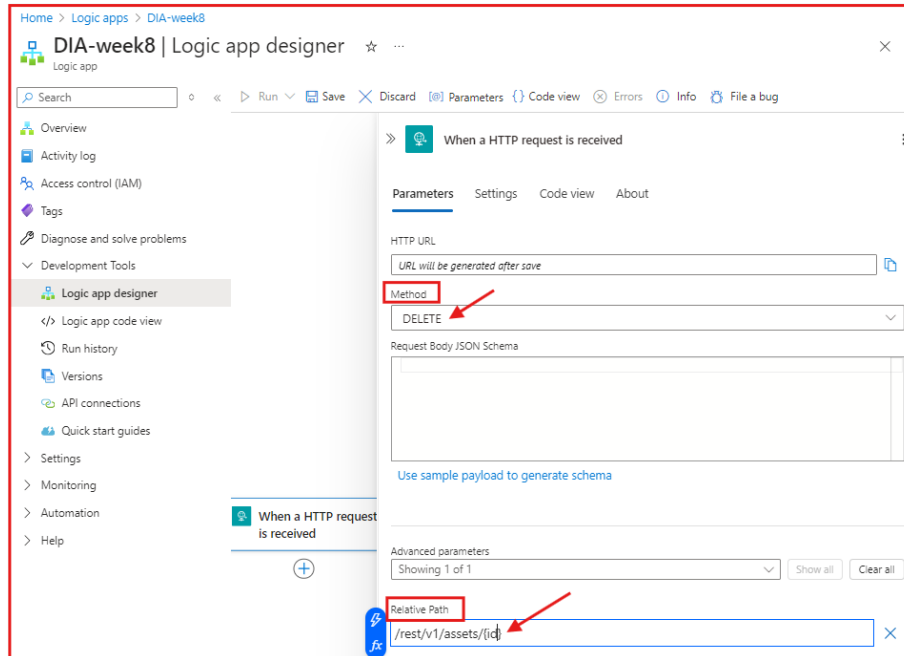


As before, note the URL of the endpoint.

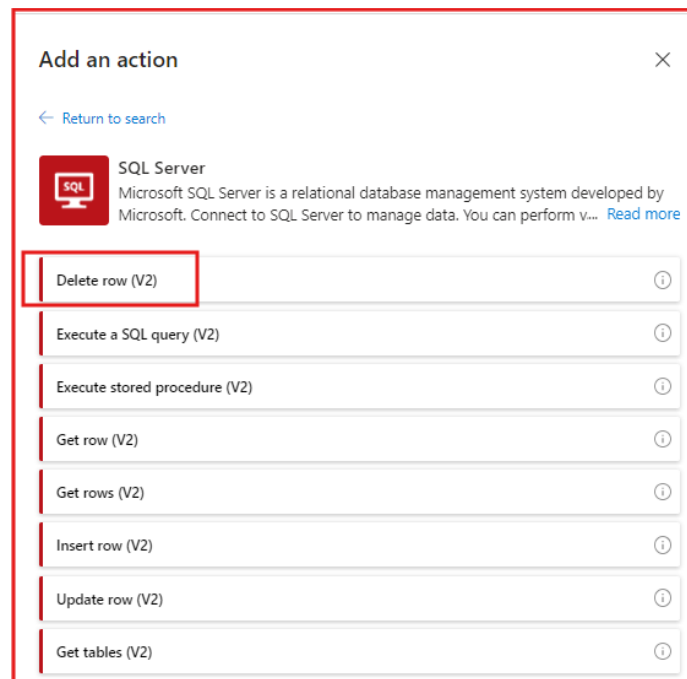
Record this in your notes, do this for all REST API endpoints.

3. Creating a DELETE endpoint (DIA)

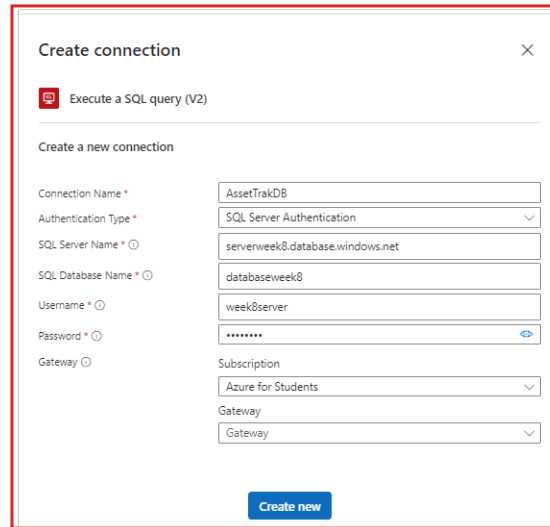
Next, create a REST endpoint for the DIA function. Start with the HTTP request trigger, as is the norm. Specify the DELETE method and the relative path, including {id}. This {id} will be interpreted as a parameter which may be reused.



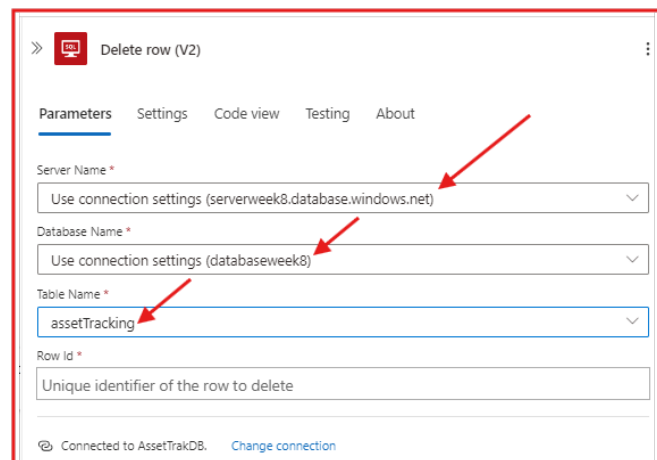
Next wire up a delete row (v2) process from the azure library.



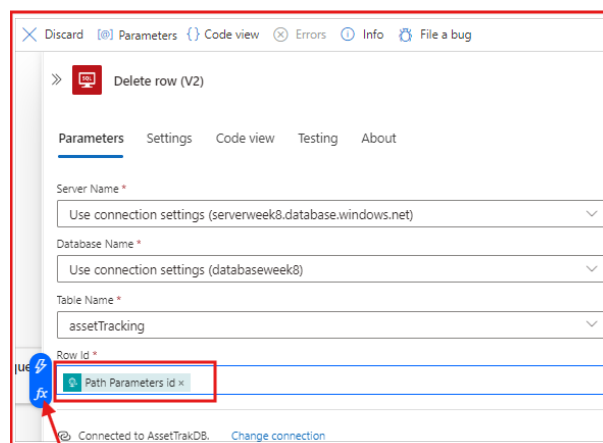
Configure the database as you have before in this tutorial.



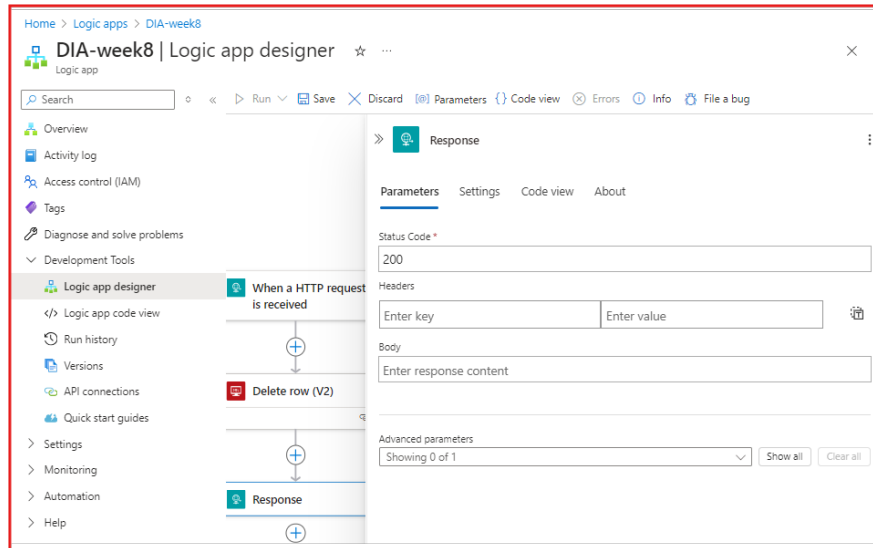
Next, specify the specifics of the SQL Delete object.



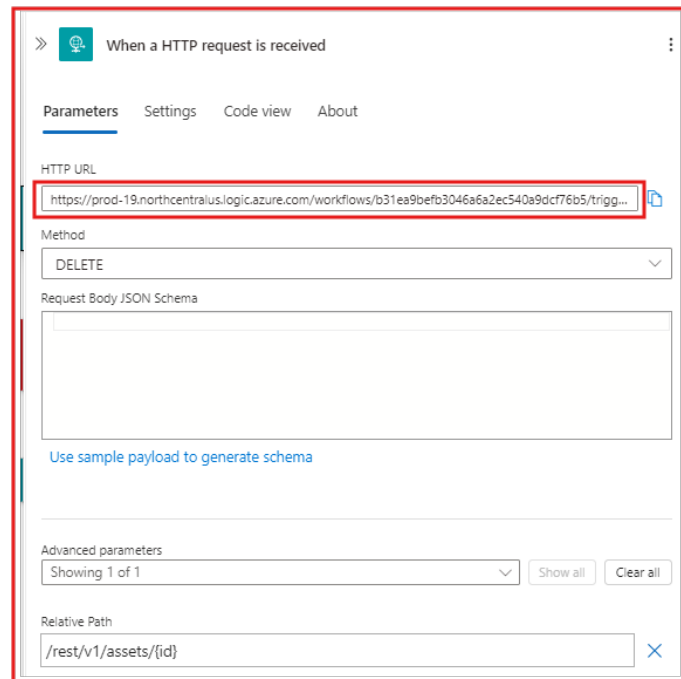
These provide the path parameter {id} / id as the row ID.



Wire up a HTTP response, as before and Save the workflow.



This is now in place. Check the URL and take a note of it.



In this case the URL was:

https://prod-19.northcentralus.logic.azure.com/workflows/b31ea9befb3046a6a2ec540a9dcf76b5/triggers/When_a_HTTP_request_is_received/paths/invoke/rest/v1/assets/%7Bid%7D?api-version=2016-10-01&sp=%2Ftriggers%2FWhen_a_HTTP_request_is_received%2Frun&sv=1.0&sig=VgRfIdSPWKXdeO_VhZZkgi37iVQn3rP4k_cVc04sysg

Note, that when this URL is to be used by clients. The {id} element **must** reflect the ID of the element one wishes to delete.

This needs to correspond to the ID in the table. In some development environments, it may be worthwhile separating this API call into two strings then using concatenation in conjunction with an ID value in order to operate correctly.

Use ‘/:id’ after /assets, as shown below, so that it will take input of value of ‘id’ from Postman.

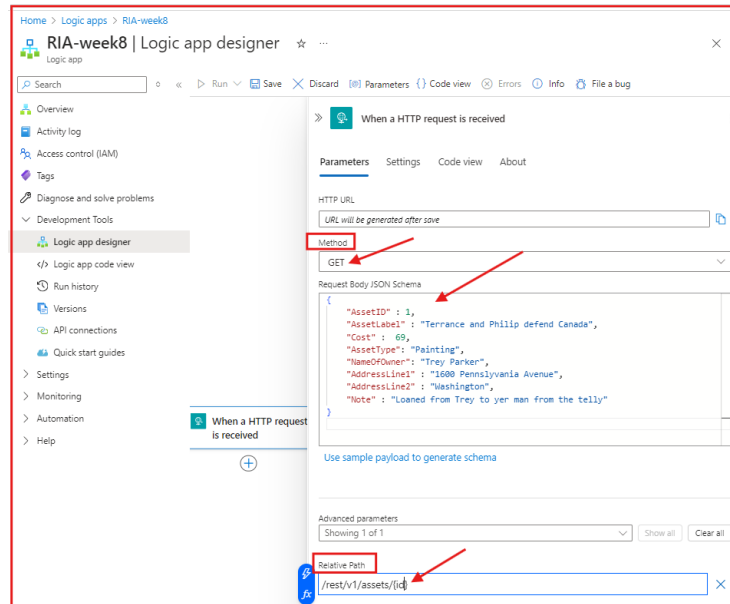
DelURI = “ https://prod-

19.northcentralus.logic.azure.com/workflows/b31ea9befb3046a6a2ec540a9dcf76b5/triggers/When_a_HTTP_request_is_received/paths/invoke/rest/v1/assets/:id”

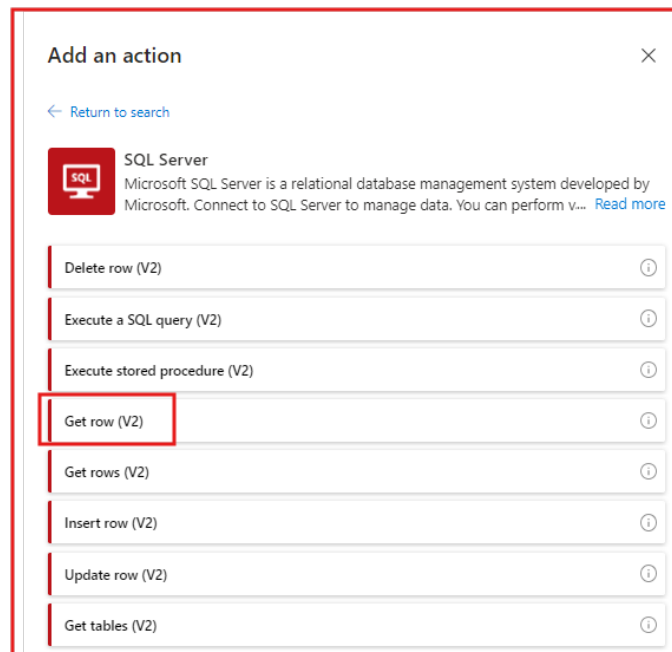
Note this overall URI in your REST API table.

4. Creating a GET endpoint to retrieve a specific record (RIA)

Create a HTTP request trigger, provide the JSON schema as before and Specify the VERB and path outlined in the design.

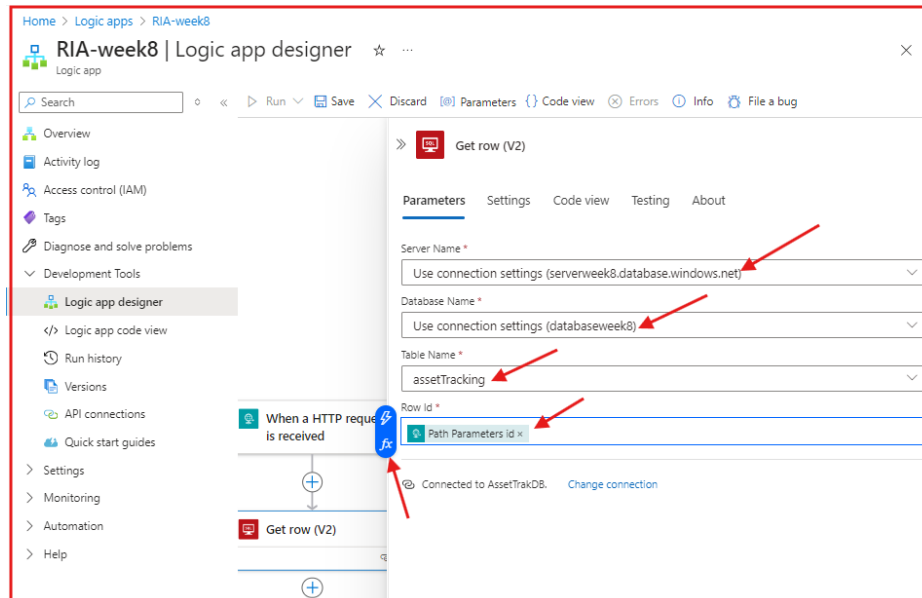


Add the Get row (v2) operation from the Azure library.

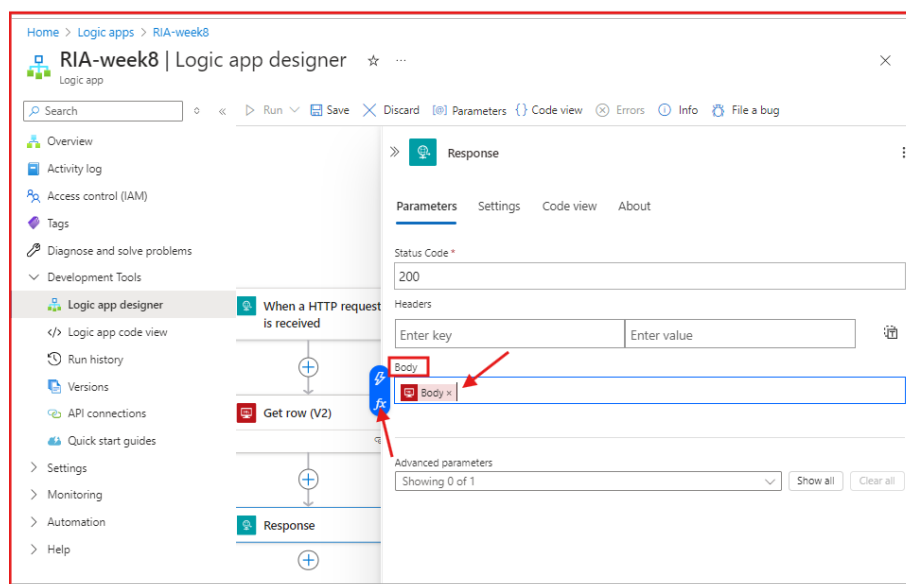


Selected the id path parameter as the Row Id for this operation to focus on. This will return a record containing that ID as its primary key.

Configure this object as you had in the previous examples.



Provide a response as described previously. In the response select the response content and set this to Body from Get Row (V2), this is the result set from the SQL query.

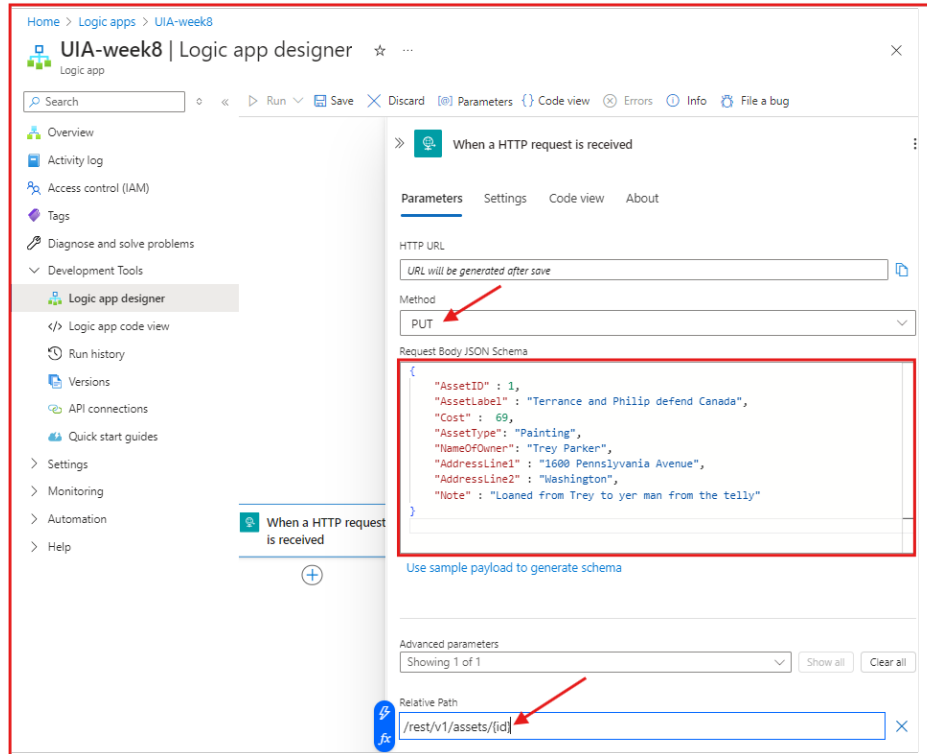


As before, note the URI of the endpoint [RIA endpoint].

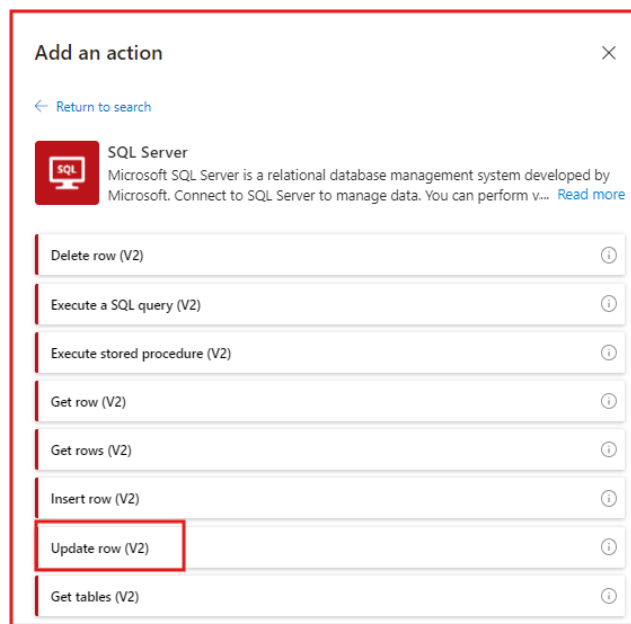
5. Creating a PUT endpoint to UPDATE a specific record (UIA)

Create a HTTP request trigger and Specify the VERB and path outlined in the design.

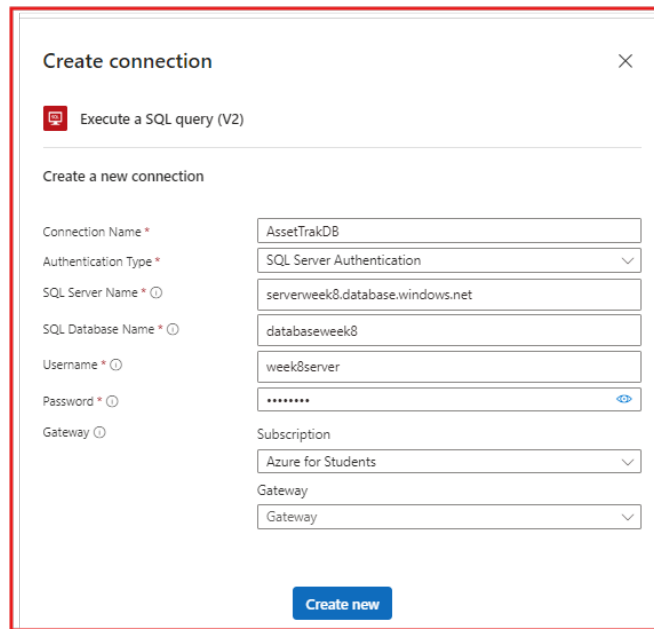
Firstly, provide a sample JSON payload. Then provide the VERB and path specification.



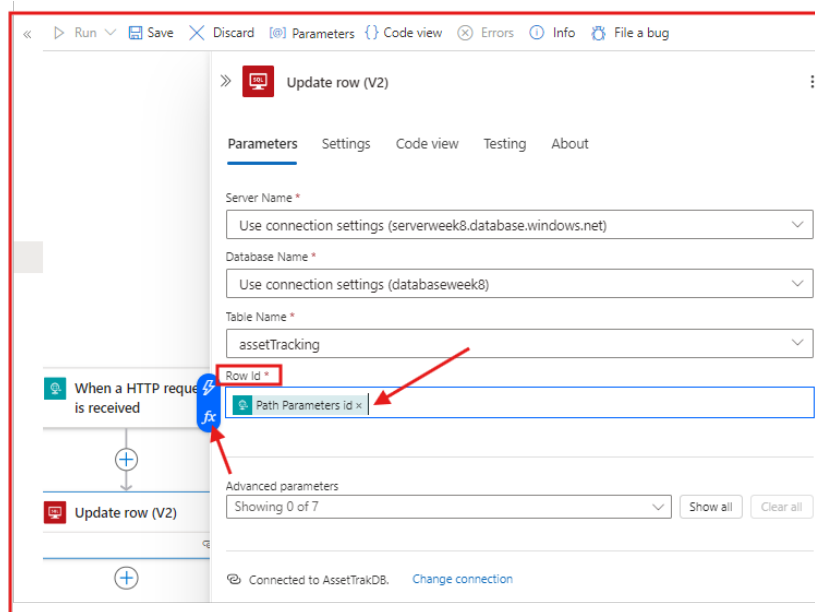
Following this, chain the SQL server Update row (V2) widget to this workflow.



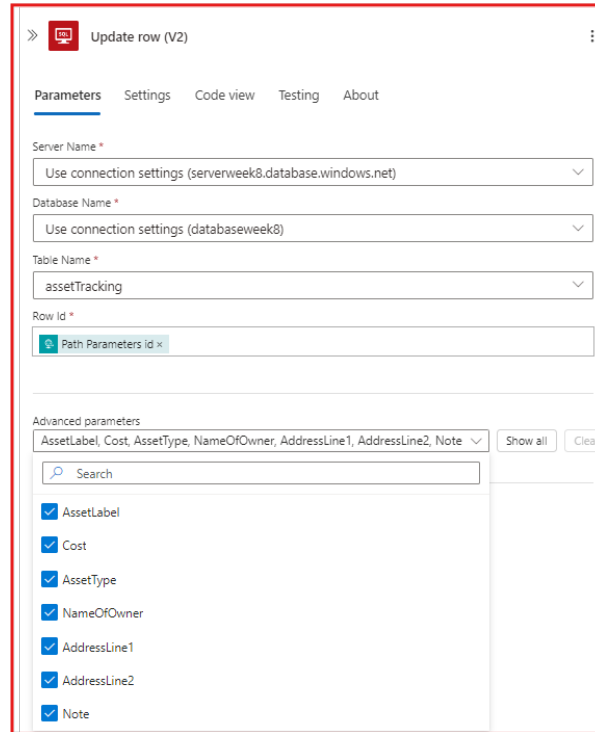
Following this, set up the database parameters as has been done many times before in this document.



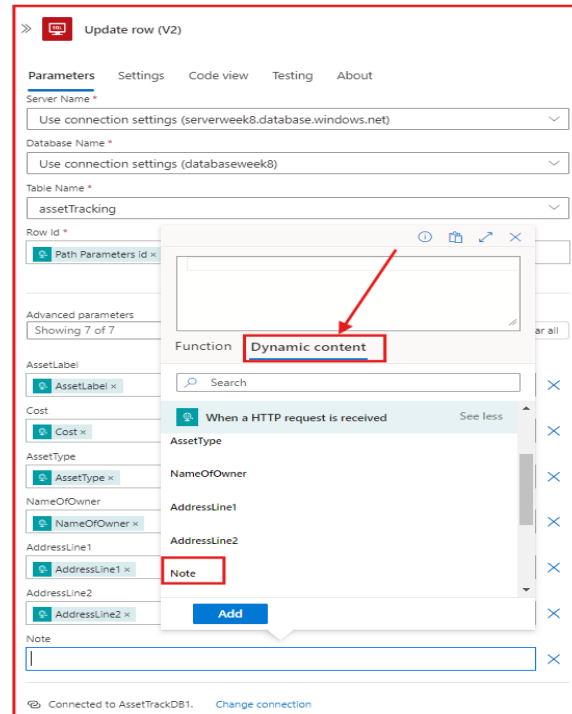
Next, set the row ID to the path parameter id.



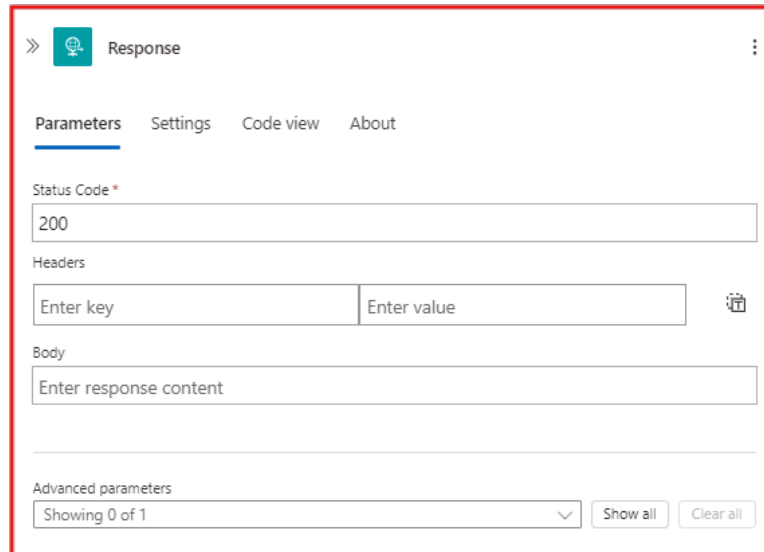
Next, add all parameters related to that database to the Update Row object.



Correspond the rows in the database to the parameters provided by the JSON document.



Next, chain on a response object as has done before.



The screenshot shows a 'Response' window with the following fields:

- Status Code ***: 200
- Headers**: A table with two columns: 'Enter key' and 'Enter value'.
- Body**: A text area with the placeholder 'Enter response content'.
- Advanced parameters**: A dropdown menu showing 'Showing 0 of 1' and buttons for 'Show all' and 'Clear all'.

Finally, note the endpoint/API URL.

In this case, it is:

https://prod-12.northcentralus.logic.azure.com/workflows/9e6b3f61cfe94dfeab685592bb70f33b/triggers/When_a_HTTP_request_is_received/paths/invoke/rest/v1/assets/%7Bid%7D?api-version=2016-10-01&sp=%2Ftriggers%2FWhen_a_HTTP_request_is_received%2Frun&sv=1.0&sig=bTsqGLF6WUPFK8r3vR2OFsfBIRQR1bnEWaGMscYQ7r0

Note this in your REST API details table. As before, this URI may need to be broken into two sub URIs for string concatenation to specify an ID.

Congratulations, you now have a full API for the assets collection.



For me, all of my 5 logic apps looks like below:

Home >

Logic apps Ulster University (ulster.ac.uk) [Get workload templates for Logic Apps](#)

+ Add [Manage view](#) [Refresh](#) [Export to CSV](#) [Open query](#) [Assign tags](#) [Enable/](#)

[You are viewing a new version of Browse experience. Click here to access the old experience.](#)

[Filter for any field...](#) [Subscription equals all](#) [Resource Group equals all](#) [Location equals all](#)

☐	Name	Status
☐	 com683imageajr	... Disabled
☐	 Week-7_Logic_App	... Disabled
☐	 Test_week7	... Disabled
☐	 CIA	... Enabled
☐	 DIA	... Enabled
☐	 IUPS	... Enabled
☐	 RAA	... Enabled
☐	 RIA	... Enabled
☐	 UIA	... Enabled

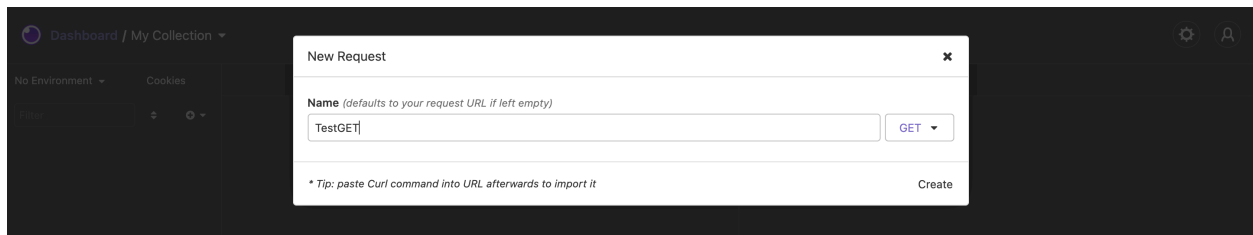
And my table with URIs looks like below:

	A	B	C	D
	Func Identifier	HTTP Verb	Relative Path	URI
2	RAA	Get	/rest/v1/assets	https://prod-20.norwayeast.logic.azure.com/workflows/34cf73755289451ebf83f86899ef4753/triggers/when_an_HTTP_request_is_received/paths/invoke?api-version=2016-10-01&sp=%2Ftriggers%2Fwhen_an_HTTP_request_is_received%2Frun&sv=1.0&sig=b-ax5EDY/147_JaLJtoQ-
3	CIA	Post	/rest/v1/assets	https://prod-24.norwayeast.logic.azure.com/workflows/fe948bc2a444b2a8f3962afcca869dd/triggers/when_an_HTTP_request_is_received/paths/invoke/rest/v1/assets?api-version=2016-10-01&sp=%2Ftriggers%2Fwhen_an_HTTP_request_is_received%2Frun&sv=1.0&sig=dZzHQ6UDZOGu9P2q5LCst3kXPQcZXlyybecj
4	DIA	Delete	/rest/v1/assets/{id}	https://prod-25.norwayeast.logic.azure.com/workflows/e5305d9597b6408fb642048ba71e6dda/triggers/when_an_HTTP_request_is_received/paths/invoke/rest/v1/assets/:id
5	RIA	Get	/rest/v1/assets/{id}	https://prod-12.norwayeast.logic.azure.com:443/workflows/1d3fef81efb7444c8347cb04724eaf62/triggers/when_an_HTTP_request_is_received/paths/invoke/rest/v1/assets/:id
6	UIA	Put	/rest/v1/assets/{id}	https://prod-01.norwayeast.logic.azure.com/workflows/48c3e8f4416b4439a0110ac06fd84c14/triggers/when_an_HTTP_request_is_received/paths/invoke/rest/v1/assets/%7Bid%7D?api-version=2016-10-01&sp=%2Ftriggers%2Fwhen_an_HTTP_request_is_received%2Frun&sv=1.0&sig=tk4bOKLIA0wldDttQ5TQnQ82jDQFLdoRLCqAE67

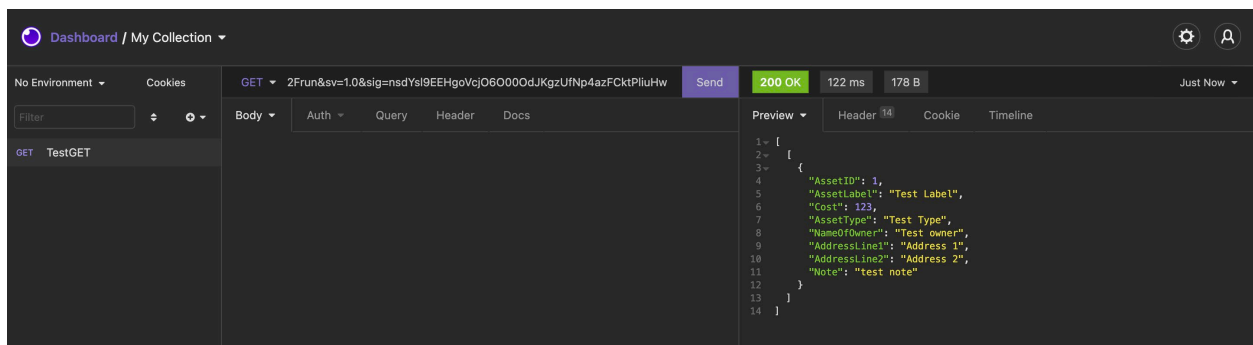
Appendix I – Testing a REST API.

The following is a graphical storyboard of testing performed on the REST APIs through use of the Insomnia tool. Other tools, such as Postman, are available.

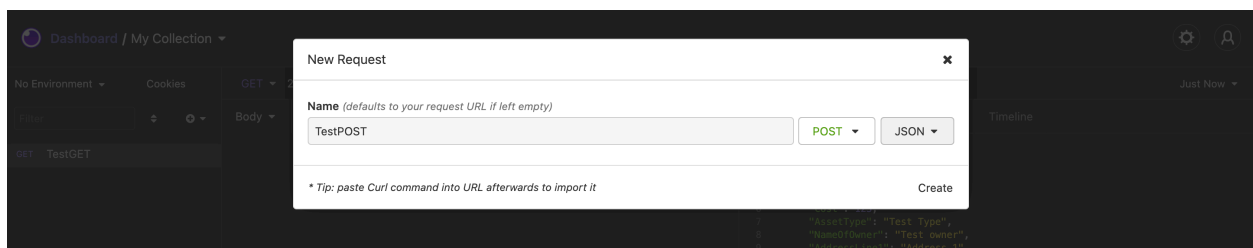
Get all assets



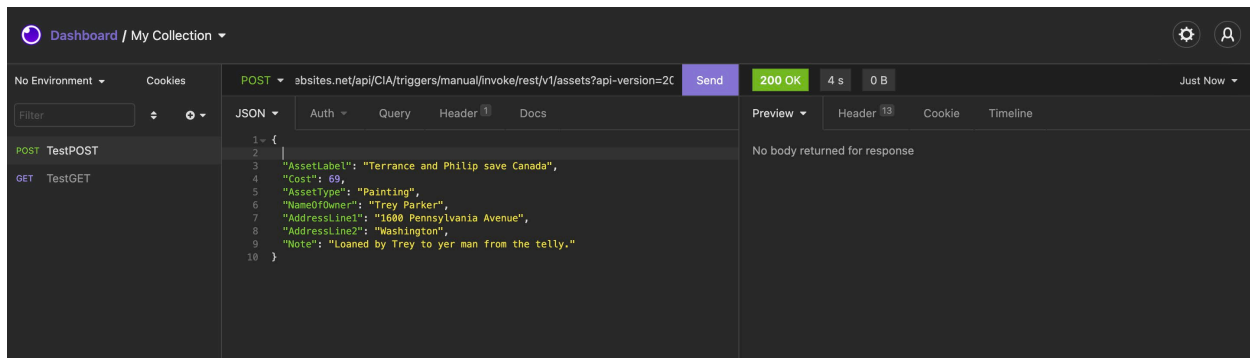
Issue a GET against the RAA URI.



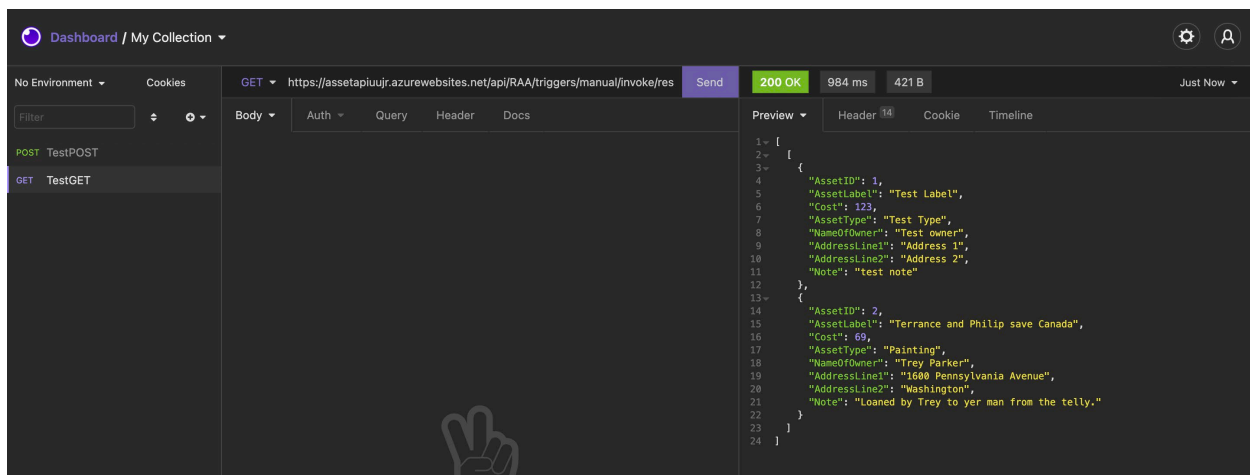
POST a new asset



Ensure you have specified the JSON content type in all operations which send a JSON record to the API endpoints.

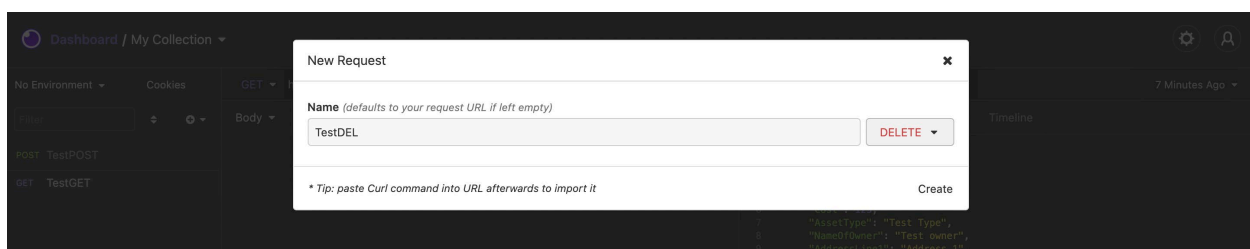


Provide the document as per the JSON spec, minus the Asset ID as it is not required and therefore not expected.



The insertion can be verified by issuing the TestGET defined previously (RAA).

DELETE a stored asset



Week-8 / Delete a stored asset

DELETE https://prod-25.norwayeast.logic.azure.com/workflows/e5305d9597b6408fb642048ba71e6dda/triggers/When_an_HTTP_request_is_received/p... Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description
api-version	2016-10-01	
sp	%2Ftriggers%2FWhen_an_HTTP_request_is_received%...	
sv	1.0	
sig	wsRpejbWV1Sm28yEw2e2Lj5iD3AejS_FhU2XXy5wyJ0	

Path Variables

Key	Value	Description
id	4	Asset ID to delete

Body Cookies Headers (20) Test Results 200 OK • 424 ms • 1004 B Save Response

Raw Preview Visualize

1

Check the assets in the database (RAA).

Dashboard / My Collection

GET <https://assetapiuljr.azurewebsites.net/api/RAA/triggers/manual/invoke/res> Send 200 OK 5.74 s 664 B Just Now

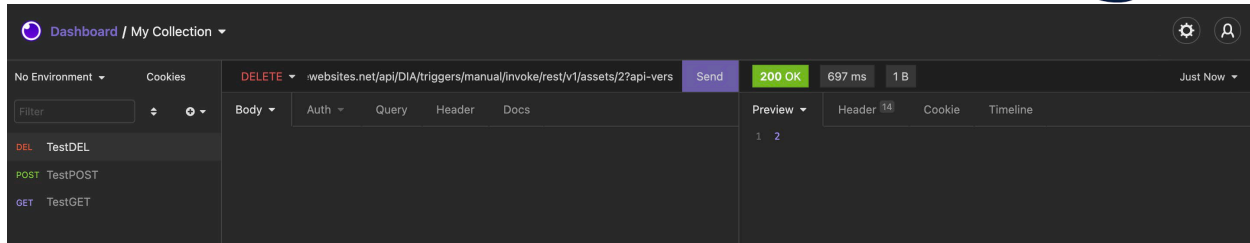
Body Auth Query Header Docs Preview Header Cookie Timeline

```

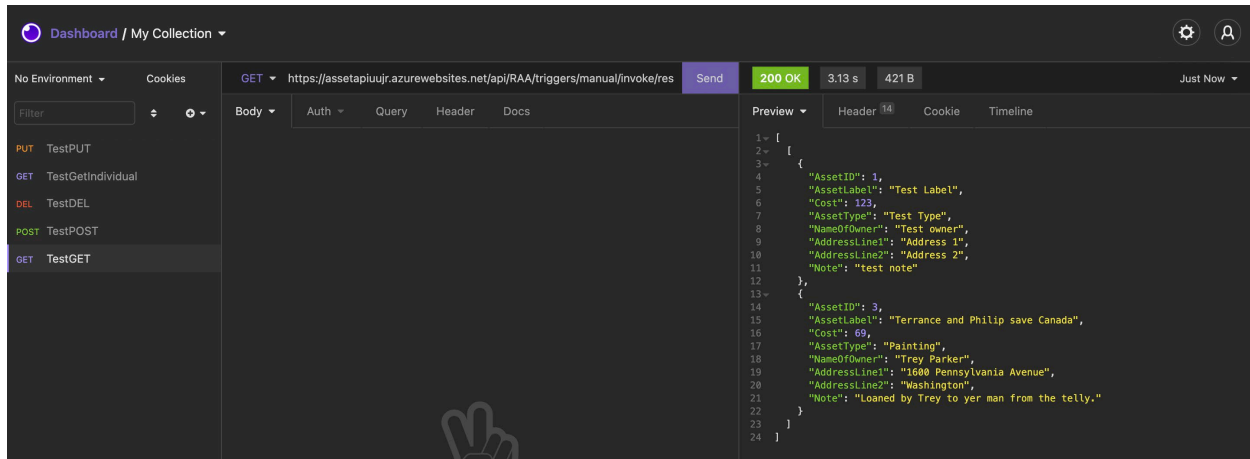
1 = [
2 = {
3 = {
4   "AssetID": 1,
5   "AssetLabel": "Test Label",
6   "Cost": 123,
7   "AssetType": "Test Type",
8   "NameOfOwner": "Test owner",
9   "AddressLine1": "Address 1",
10  "AddressLine2": "Address 2",
11  "Note": "test note"
12 },
13 {
14  "AssetID": 2,
15  "AssetLabel": "Terrance and Philip save Canada",
16  "Cost": 69,
17  "AssetType": "Painting",
18  "NameOfOwner": "Trey Parker",
19  "AddressLine1": "1600 Pennsylvania Avenue",
20  "AddressLine2": "Washington",
21  "Note": "Loaned by Trey to yer man from the telly."
22 },
23 {
24  "AssetID": 3,
25  "AssetLabel": "Terrance and Philip save Canada",
26  "Cost": 69,
27  "AssetType": "Painting",
28  "NameOfOwner": "Trey Parker",
29  "AddressLine1": "1600 Pennsylvania Avenue",
30  "AddressLine2": "Washington",
31  "Note": "Loaned by Trey to yer man from the telly."
32 },
33 }
34 ]
  
```

Select a body type from above

Note this is a Delete activity executed against the DIA URI. The Asset {id} needs to correspond to an item in the collection, in this case item 2. The response from the server simply details the deleted ID.

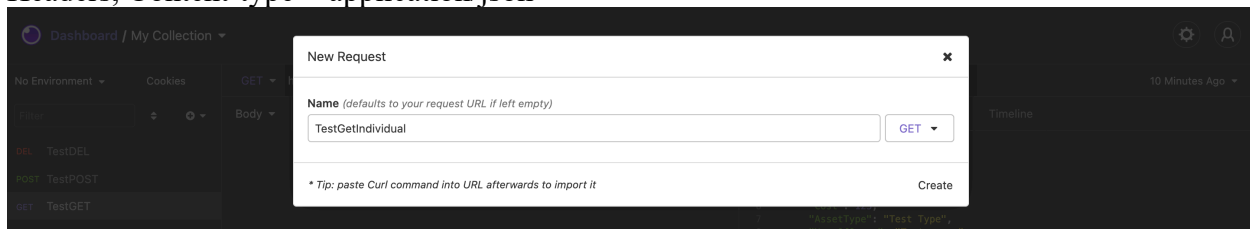


Check the assets in the database (RAA) to confirm the deletion was a success.

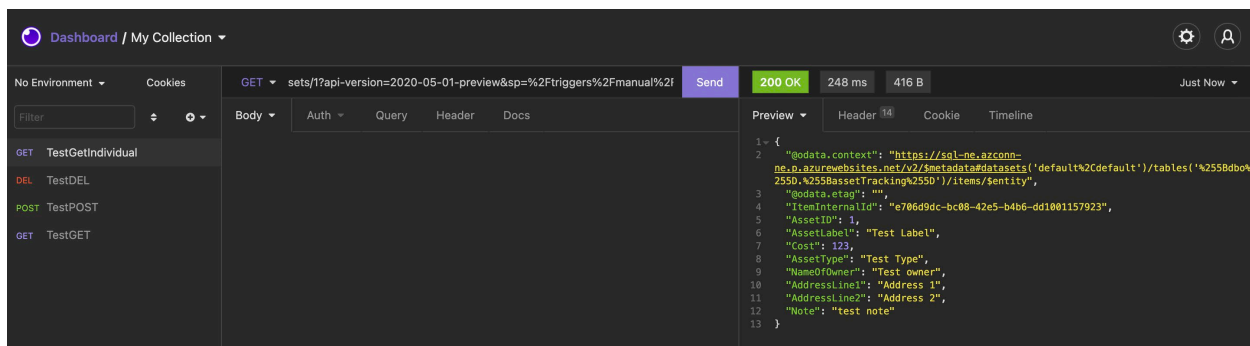


GET an individual record

Headers, Content-type = application/json

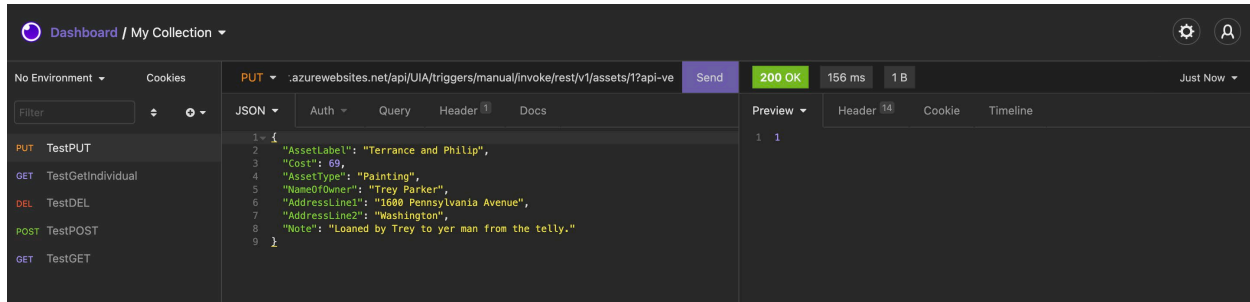


Issue a get against the ID (RIA URI) of an asset of interest.

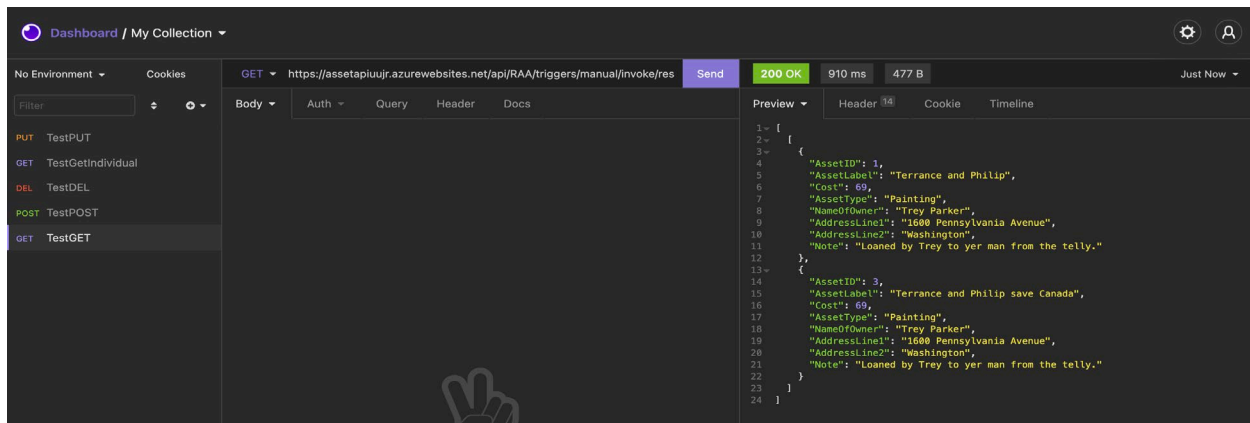


PUT/Update an individual record

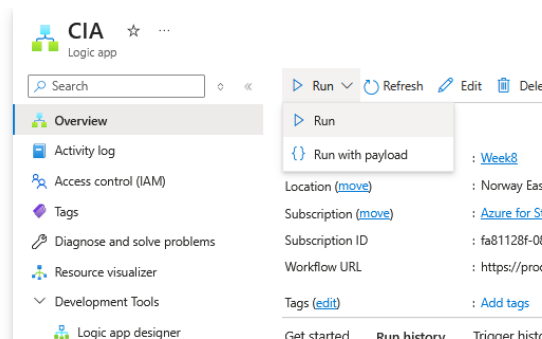
Asset 1 doesn't have the correct details, issue a PUT against its (UIA) URL provide a new JSON document to provide the changes (minus AssetID).



Check this change was enacted (RAA).



If you find any error during testing, go to Azure portal, click on Logic App you want to test, click on 'Run with Payload' to test at portal.



You do not need to delete these Logic Apps, instead 'diabie' them as they will be used in next lab.